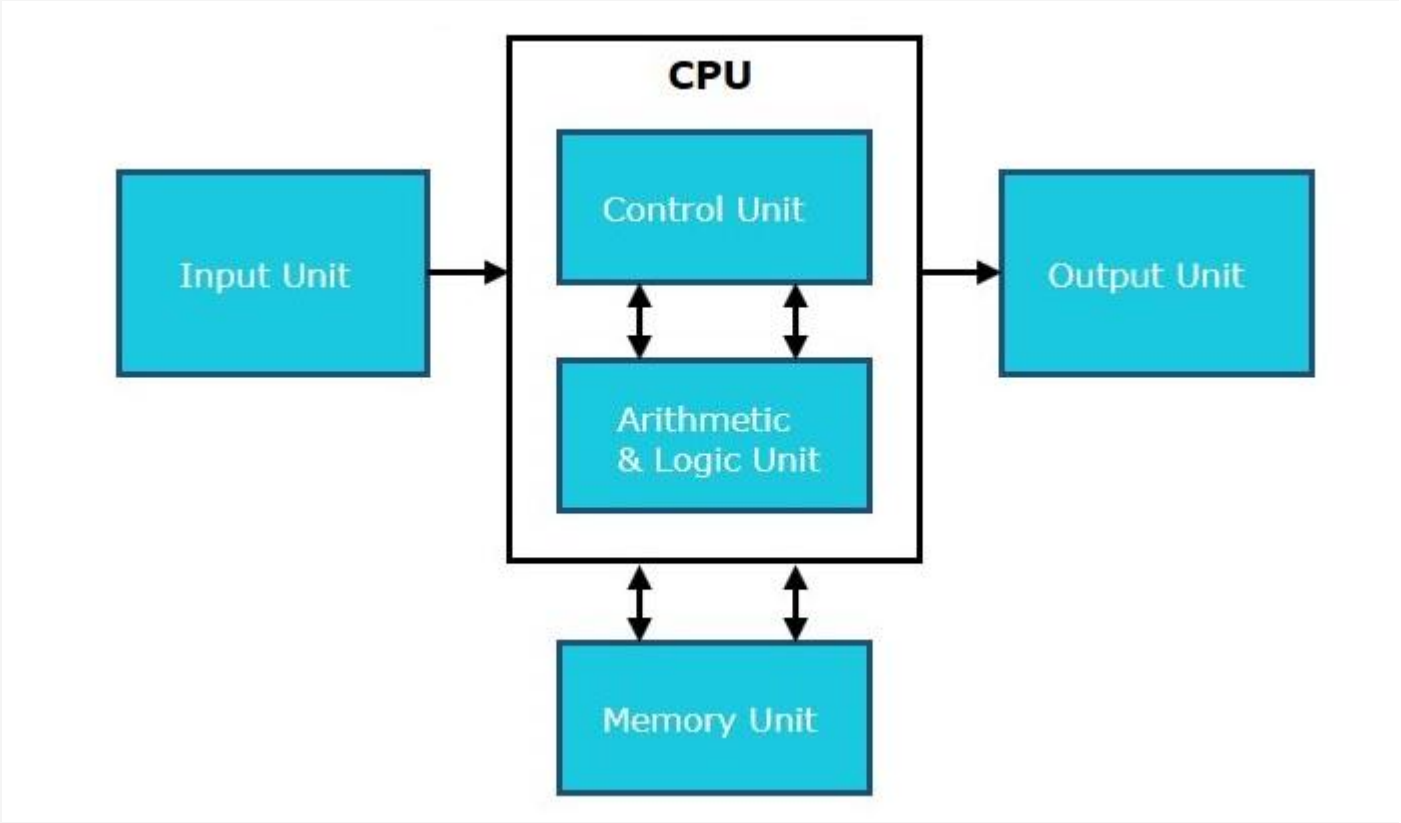
MODULE 1

Fundamentals of Logic System Design

Functional units of a computer

- Input Unit
- Output Unit
- Central processing Unit (ALU and
- Control Units)
- Memory



FUNCTIONAL UNITS

1. **INPUT UNIT:**

This unit is responsible for receiving data and instructions from the outside world, such as from a keyboard, mouse, or other input devices.

- It converts this input into a format that the computer can understand(binary format) which can be understood by CPU
- Eg: Keyboard,Mouse, Joystick,microphone(audio signal-electrical signal)

2 OUTPUT UNIT:

- The output unit sends the processed results to the outside world, typically through devices like monitors, printers, or speakers.
- It converts the computer's internal representation of the data into a form that humans can understand.

- Eg: Monitor, Printer, LCD, LED etc

3

ARITHMATIC AND LOGIC UNIT (ALU):

- Fundamental component of a computer's Central Processing Unit (CPU).
- It's responsible for performing arithmetic and logical operations on binary data

FUNCTIONAL UNITS

4 **MEMORY UNIT:**

- The memory unit stores data and instructions, both temporarily (like during processing) and permanently (like on a hard drive).
- It can be divided into primary memory (RAM) and secondary memory (like hard drives and SSDs).
- **Primary Memory**
- RAM is also known as primary or temporary memory; it is a type of
- volatile memory used for temporarily storing data.
- The contents inside the RAM are erased when the computers power gets off or restarted.

4

MEMORY UNIT:

- **Storage (Hard Drives, SSDs, Flash Drives, etc.)**
- Storage devices are used to store the data permanently, even when the computer is powered off.
- They are non-volatile; the data remains intact even when the power is turned off or the system restarts.
- The most popular and commonly used storage devices are Hard disks (HDs), Solid-State Drives (SSDs), USB flash drives, and optical disks (e.g., DVDs), pen drives.

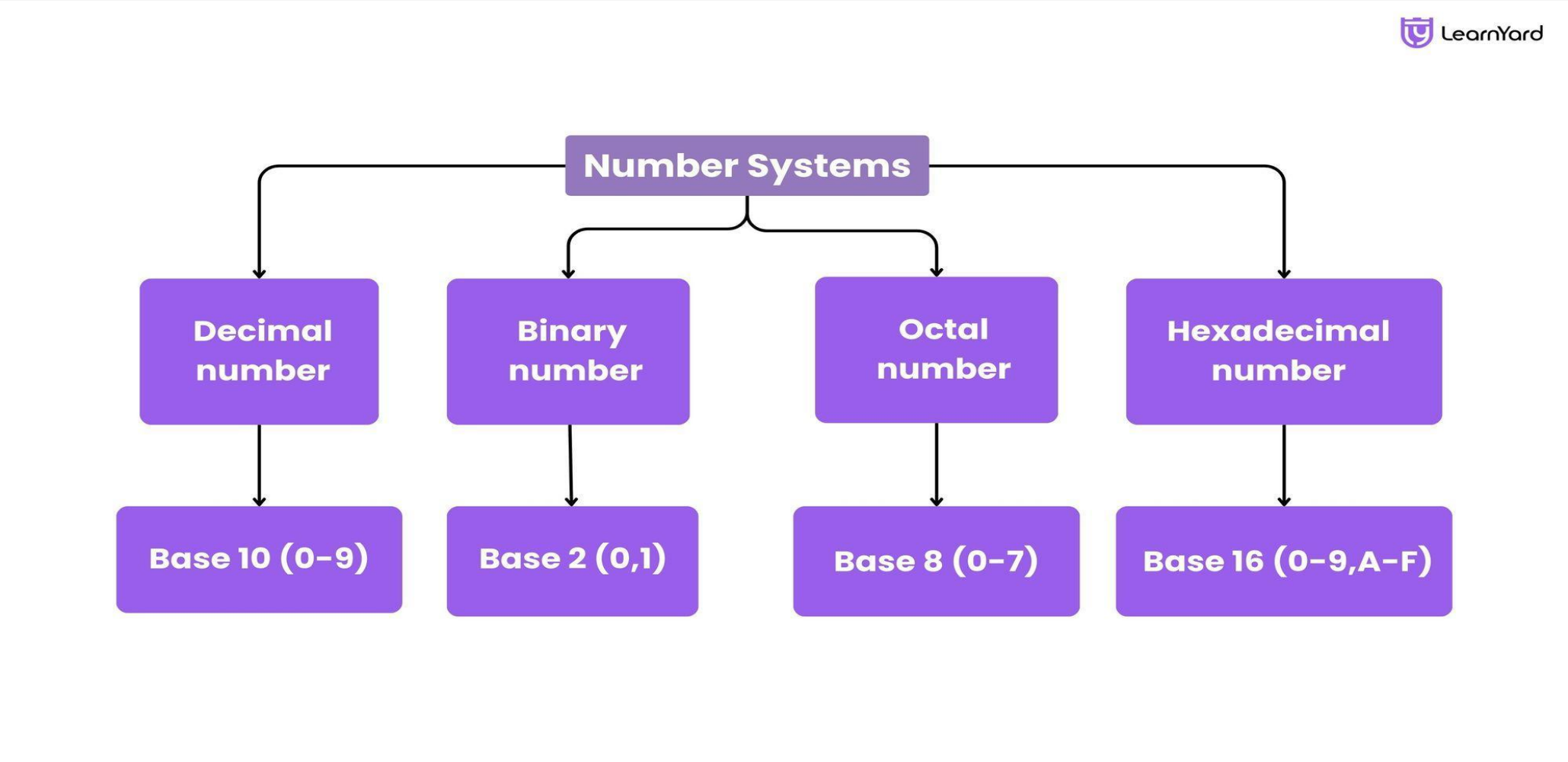
5 **CONTROL UNIT:**

• The Control Unit is the part of the computer's central processing unit (CPU), which directs the operation of the processor .

• It fetches instructions from memory, decodes them, and generates control signals to execute these instructions, effectively managing the flow of data and operations within the CPU

- **Functions of Control Unit**
- **Instruction Fetch** − A CU fetches instructions from RAM (Random Access Memory).
- **Instruction Decoding** − It decodes the fetched instructions to operate.
- **Instruction Execution** − A CU sends control signals to perform operations like ALU for arithmetic and logical operations.
- **Control Flow Management** − It controls flow by updating the programme counter.

Number Systems



CONVERTION

1 DECIMAL TO ANY OTHER BASE.

A) Divide the Number (Decimal Number) by the base of target base system (in which you want to convert the number: Binary (2), octal (8) and Hexadecimal (16)).

B)The binary equivalent is the remainders read from bottom to top

For Fractional Part:

- Multiply the fractional part base of target base system (in which you want to convert the number: Binary (2), octal (8) and Hexadecimal (16)).
- Record the integer part (0 or 1).
- Take the fractional part of the result and repeat the multiplication.
- Continue until the fractional part becomes 0 or reaches the desired

Decimal to Binary Conversion				Result
Decimal Number is : $(12345)_{10}$				
2	12345	1	LSB	
2	6172	0		
2	3086	0		
2	1543	1		
2	771	1		
2	385	1		
2	192	0		Binary Number is $(11000000111001)_2$
2	96	0		
2	48	0		
2	24	0		
2	12	0		
2	6	0		
2	3	1		
	1	1	MSB	

CONVERTION

1 DECIMAL TO ANY OTHER BASE.

Decimal to Octal Conversion				Result
Decimal Number is : $(12345)_{10}$				
8	12345	1	LSB	Octal Number is $(30071)_8$
8	1543	7		
8	192	0		
8	24	0		
	3	3	MSB	

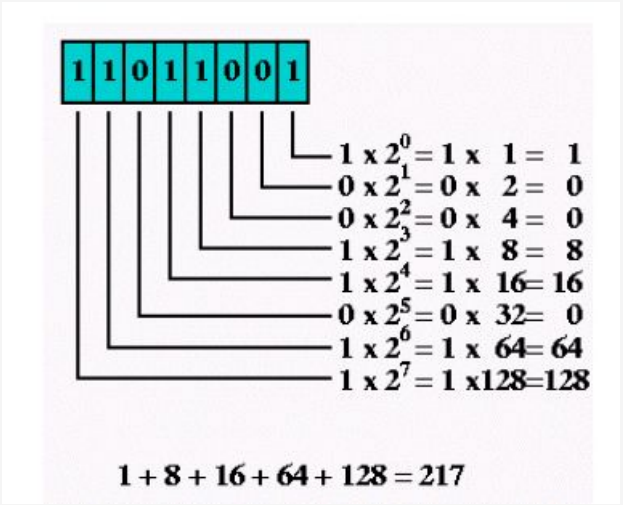
Decimal to Hexadecimal Conversion				Result
Example 1 Decimal Number is : $(12345)_{10}$				
16	12345	9	LSB	Hexadecimal Number is $(3039)_{16}$
16	771	3		
16	48	0		
8	3	3	MSB	
Example 2 Decimal Number is : $(725)_{10}$				Hexadecimal Number is $(2D5)_{16}$ **Convert** **10, 11, 12, 13, 14, 15** **to its equivalent...** **A, B, C, D, E, F**
16	725	5	LSB	
16	45	13	D	
	2	2	MSB	

CONVERTION

2 BINARY TO OTHER NUMBER SYSTEM.

BINARY TO DECIMAL.

- Multiply each digit by 2 raised to the power of its position, starting from 0 (rightmost digit).
- Add up the results of these multiplications.
- The sum is the decimal equivalent of the binary integer.
- **For Fractional Part:**
- Multiply each digit by 2 raised to the negative power of its position, starting from -1 (first digit after the decimal point).
- Add up the results of these multiplications.
- The sum is the decimal equivalent of the binary fraction.



Example: $(1010.01)_2$

$1x2^3 + 0x2^2 + 1x2^1 + 0x2^0 + 0x2^{-1} + 1x2^{-2} = 8+0+2+0+0+0.25 = 10.25$

Thus, $(1010.01)_2 = (10.25)_{10}$

CONVERTION

2 BINARY TO OTHER NUMBER SYSTEM.

Binary to octal.

- Starting from the rightmost bit, divide the binary number into groups of 3 bits.
- If the number of bits is not a multiple of 3, add leading zeros to the leftmost group.
- Each 3-bit binary group corresponds to a single octal digit.

Octal	Binary Equivalent
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

CONVERTION

2 Binary to octal.

Binary: 11100101 =	11 100 101	
	011 100 101	Pad the most significant digits with zeros if necessary to complete a group of three.

Binary =	011	100	101	
Octal =	3	4	5	= 345 oct

Example: $(111101101)_2$

$$\underline{111}\ \underline{101}\ \underline{101}$$
$$\ \ |\ \ \ |\ \ \ |$$
$$7\ \ 5\ \ \ 5$$

Thus, $(111101101)_2 = (755)_8$

CONVERTION

2 Binary to hexadecimal

Group binary digits into sets of four, starting with the least significant (rightmost) digits.

Binary: 11100101 = 1110 0101
Then, look up each group in a table:

Binary =	1110	0101	
Hexadecimal =	E	5	= E5 hex

Binary:	0000	0001	0010	0011	0100	0101	0110	0111
Hexadecimal:	0	1	2	3	4	5	6	7

Binary:	1000	1001	1010	1011	1100	1101	1110	1111
Hexadecimal:	8	9	A	B	C	D	E	F

CONVERTION

3

• OCTAL TO DECIMAL

For Integer Part:

•Write down the octal number.

•Multiply each digit by 8 raised to the power of its position, starting from 0 (rightmost digit).

•Add up the results of these multiplications.

•The sum is the decimal equivalent of the octal integer.

For Fractional Part:

•Write down the octal fraction.

•Multiply each digit by 8 raised to the negative power of its position, starting from -1 (first digit after the decimal point).

•Add up the results of these multiplications.

•The sum is the decimal equivalent of the octal fraction.

Example: $(12.2)_8$

$1 \times 8^1 + 2 \times 8^0 + 2 \times 8^{-1} = 8+2+0.25 = 10.25$

Thus, $(12.2)_8 = (10.25)_{10}$

345 octal = $(3 * 8_2) + (4 * 8_1) + (5 * 8_0) = (3 * 64) + (4 * 8) + (5 * 1) = 229$ decimal

NUMBER REPRASENTATION

Computers use **binary number systems** to represent all kinds of data, including **integers** and **real (floating-point)** numbers.

INTEGER NUMBER SYSTEM

1:Signed Integer Representation

2:Unsigned Integer Representation

UNSIGNED INTIGER REPRESENTATION

The bits present in the un-signed binary number holds the magnitude of a number.

That means, if the un-signed binary number contains ‘N’ bits, then all N bits represent the magnitude of the number, since it doesn’t have any sign bit.

Example

Consider the decimal number 108. The binary equivalent of this number is 1101100. This is the representation of unsigned binary number.

(108)10 = (1101100)2 It is having 7 bits.

These 7 bits represent the magnitude of the number 108

SIGNED INTIGER REPRESENTATION

There are three types of representations for signed binary numbers

- Sign-Magnitude form
- 1’s complement form
- 2’s complement form

Sign-Magnitude form

The Most Significant Bit MSB of signed binary numbers is used to indicate the sign of the numbers. Hence, it is also called as sign bit.

The positive sign is represented by placing ‘0’ in the sign bit.

The negative sign is represented by placing ‘1’ in the sign bit.

If the signed binary number contains ‘N’ bits, then N–1 bits only represent the magnitude of the number since one-bit MSB is reserved for representing sign of the number.

1's complement form

1. For +ve numbers the representation rules are the same as signed integer

- For +ve numbers the representation rules are the same as signed integer representation.

  For –ve numbers,

- Write the +ve number in binary and take 1's complement of it.

For *1's complement of 0 = 1 and 1's complement of 1 = 0*

Example:

(-5) in 1's complement:

+5 = 0101

-5 = 1010

The range of 1's complement integer representation of n-bit number is given as $-(2^{n-1} - 1)$ to $2^{n-1} - 1$.

- –ve numbers, we can follow any one of the two approaches:

•Write the +ve number in binary and take 1's complement of it.

1’s complement form

1. For +ve nu

Example-3: Find 1’s complement of each 3 bit binary number.

Simply invert each bit of given binary number, so 1’s complement of each 3 bit binary number will be,

Binary number	1's complement
000	111
001	110
010	101
011	100
100	011
101	010
110	001
111	000

2's Complement representation

In 2's Complement representation the following rules are used:

1. For +ve numbers, the representation rules are the same as signed integer representation.
2. For –ve numbers

To take 2's complement simply take 1's complement and add 1 to it.

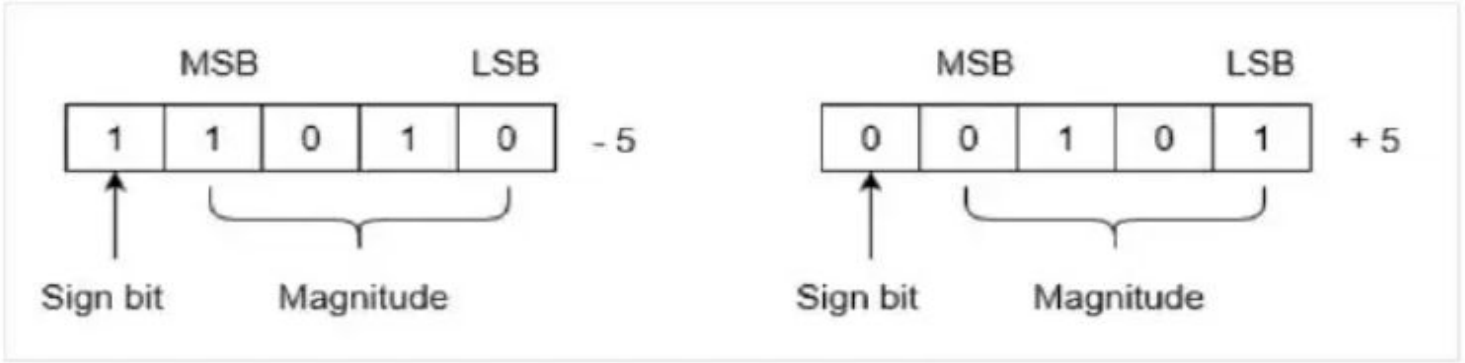
Example:

(-5) in 2's complement
(+5) = 0101
1's complement of (+5) = 1010
Add 1 in 1010: 1010+1 = 1011
Therefore (-5) = 1011

2's Complement representation

1. For +ve nu
plement of it.

Example: Let we are using 5 bits register. The representation of -5 and +5 will be as follows:



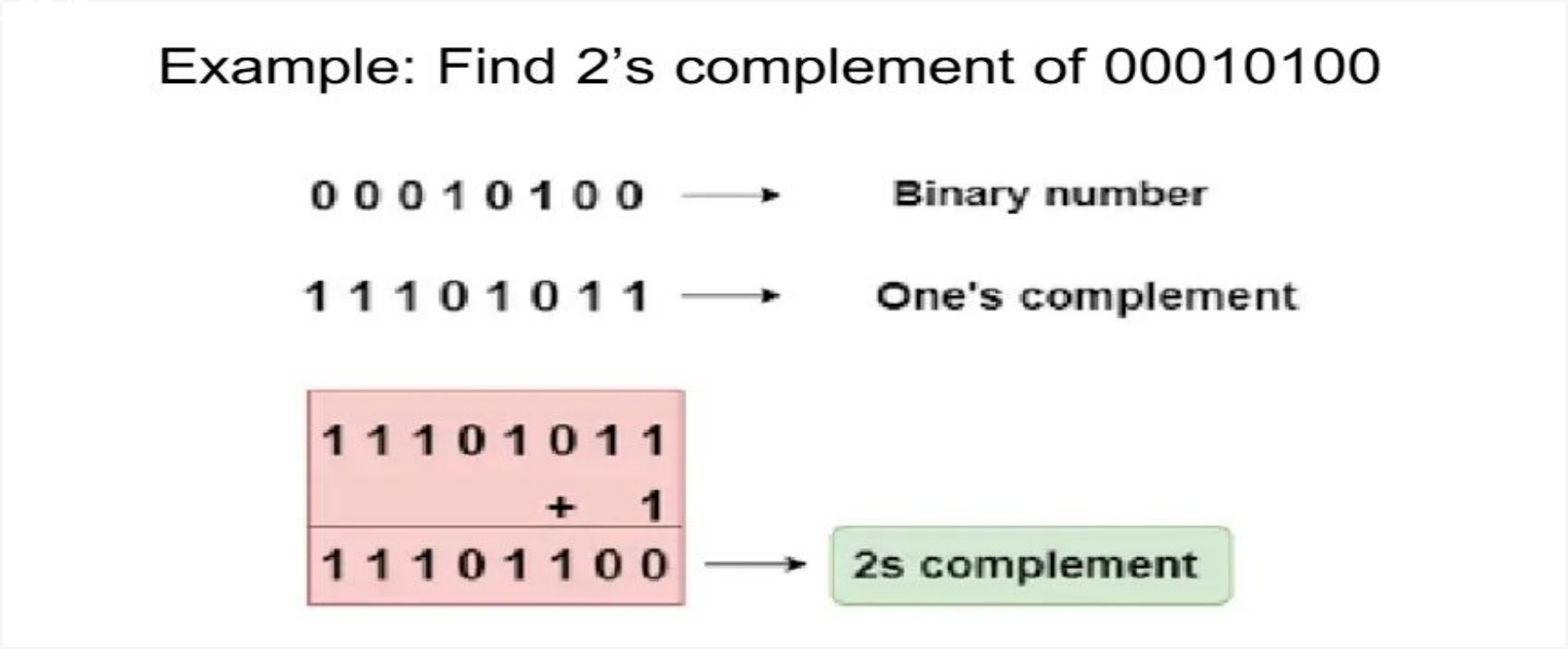
+5 is represented as it is represented in sign magnitude method. -5 is represented using the following steps:

(i) +5 = 0 0101

(ii) Take 1's complement of 0 0101 and that is 1 1010. MSB is 1 which indicates that number is negative.

2’s Complement representation

1. For +ve numbers the representation rules are the same as signed integer
plement of it



FLOATING POINT REPRESENTATION

1. For +ve numbers the representation rules are the same as signed integer

- Floating-point representation is a method of storing **real numbers (like 3.14, -0.01, 2.718, etc.)** in a way that supports a **wide range of values** (very big or very small), using **scientific notation**.

- A floating-point number has **three parts**:

  Sign bit (S):

  0 → positive number

  1 → negative number

  Exponent (E):

  Shows the position of the decimal point

  Stored using **"bias"** (to allow both negative and positive exponents)

  Mantissa (M) (or **Significand**):

  The actual digits of the number (after the decimal point)

FLOATING POINT

General Formula:

$$\text{Value} = (-1)^S \times 1.M \times 2^{(E - \text{Bias})}$$

The "1." is **implied** in normalized form (for non-zero numbers)

FLOATING POINT –REPRESENTATION –IEEE STANDARD

2 Type representation

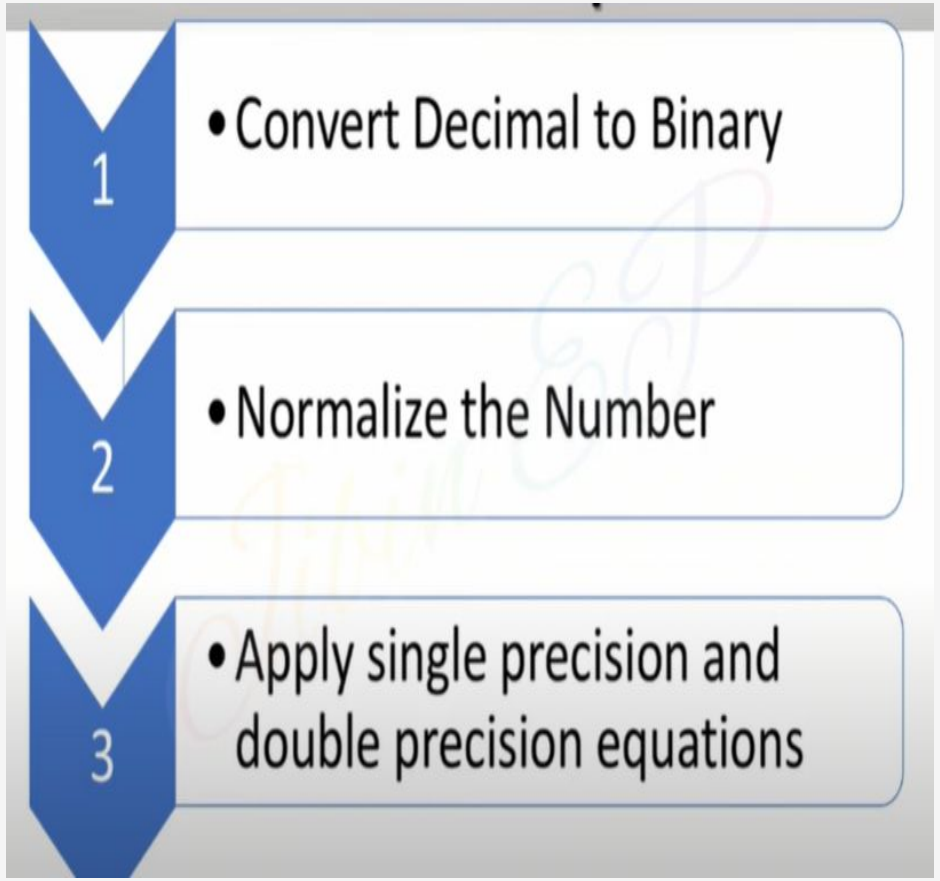
1: Single Precision Method – 32 bit considered

sign exponent Mantisa

2: Doble precision Method – 64 bit is considered

sign Exponent Mantisa

STEPS



Equations for Single and Double Precisions

Single Precision

$$(1.N)2^{E-127}$$

Double Precision

$$(1.N)2^{E-1023}$$

Example

Represent $(1259.125)_{10}$ in single and double precision Format

Representing +5.75 in Single and double precision format

Sign bit (S) = 0 (positive)
Exponent = 2 ⟶ Stored as 2 + 127 = **129 = 10000001**
Mantissa = 01110000000000000000000 (only 23 bits)

Double Precision (64 bits)
Representing -6.25

BOOLEAN ALGEBRA

- Boolean algebra :Boolean algebra is the branch of algebra that deals with logical operations and binary variables.

- It operate only two numbers 0 and 1

1. For +ve numbers the representation rules are the same as signed integer plement of it.

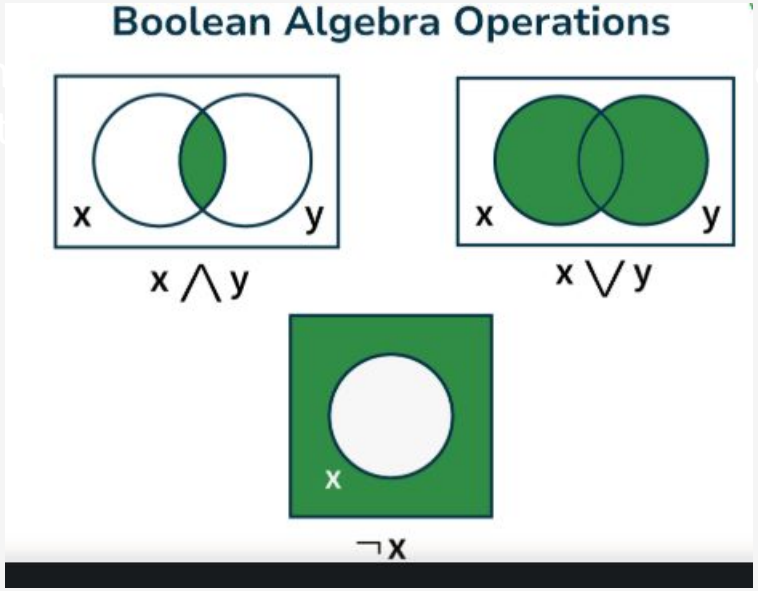
- Binary variable can take either of two values 0 and 1(TRUE or FALSE)

- It used to analyze and simplify the digital circuit

OPERATION –BOOLEAN ALGEBRA

- ***Negation*** *or NOT Operation (complement –)*
- ***Conjunction*** *or AND Operation(dot sign .)*
- ***Disjunction*** *or OR Operation(Plus sign +)*

ues, both very small and very large. The **single precision** format uses 32 bits, while **double pr**



1. For +ve n e the same as signed integer
   plement of it

OPERATION –BOOLEAN ALGEBRA

- ***Negation*** *or NOT Operation*

ues, bo ses 32 bits, while **double pr**

- *If A = 1, then using NOT operation we have (A)' = 0*
- *If A = 0, then using the NOT operation we have (A)' = 1*
- *We also represent the negation operation as ~A, i.e if A = 1, ~A = 0*

Conjunction or AND Operation

- *f A = True, B = True, then A . B = True* *A.A=A*
- *If A = True, B = False, Or A = false, B = True, then A . B = False* *A.0=0*
- *If A = False, B = False, then A . B = False* *A.1=A*

Disjunction (OR) Operation **LAW**

If A = True, B = True, then A + B = True *A + A=A*
If A = True, B = False, Or A = false, B = True, then A + B = True *A+0=A*
If A = False, B = False, then A + B = False *A+1=1*

Boolean Algebra Table

Operation	Symbol	Definition
AND Operation	· or ∧	Returns true only if both inputs are true.
OR Operation	+ or ∨	Returns true if at least one input is true.
NOT Operation	¬ or ~	Reverses the input.
XOR Operation	⊕	Returns true if exactly one input is true.
NAND Operation	↓	Returns false only if both inputs are true.
NOR Operation	↑	Returns false if at least one input is true.
XNOR Operation	↔	Returns true if both inputs are equal.

Boolean Algebra Laws

Identity Laws

Identity elements for both AND(.) and OR(+) oprations

The Boolean algebrae have such variables that on operating with AND and OR operation,we get the result is

A+0=A

A.0=A

Commutative Law

- Binary variables in Boolean Algebra follow the commutative law.
- This law states that operating boolean variables A and B is similar to
- operating boolean variables B and A. That is,
- $A.\ B = B.\ A$

Boolean Algebra Laws

Associative Laws:

- These Law allow grouping of the Boolean variable
- A+(B+C)=(A+B)+C
- A.(B.C)=(A.B).C

Distributive Law:

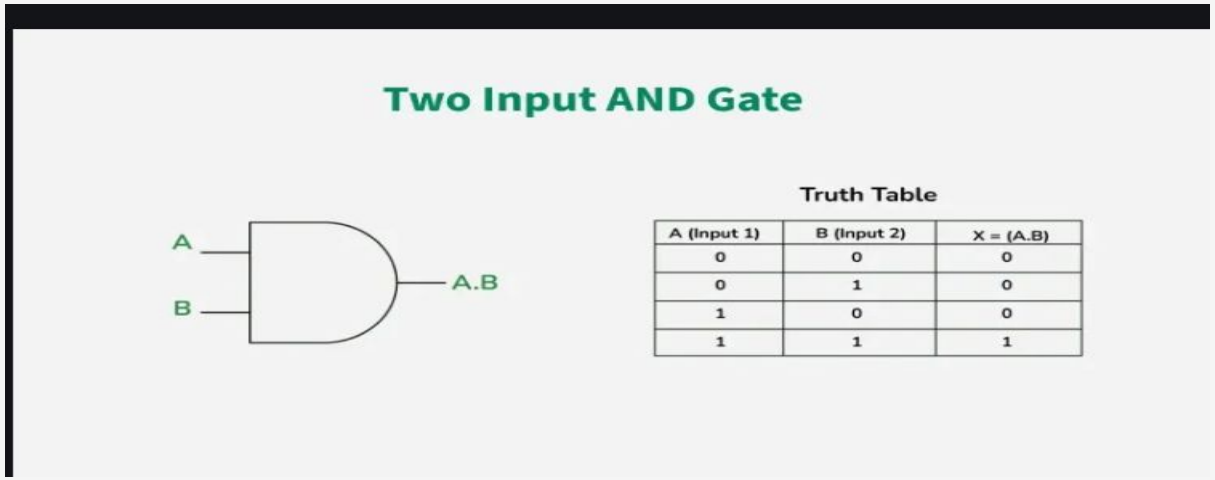
- The distributive law is written for three variables as follows:
- A(B + C) = AB + AC
- This law states that ORing two or more variables and then ANDing the result with a single variable is equivalent to ANDing the single variable with each of the two or more variables and then ORing the products.

GATES

- Logic gates are the fundamental building blocks of digital circuits.
- They manipulate binary inputs (0s and 1s) to produce a single binary output based on specific logical operations.
- like switches that turn on or off depending on the input signals.
- Truth tables are used to represent and understand the behavior of logic gates
- There are basically seven main types of logic gates that are used to perform various logical operations in digital systems.
- By combining different logic gates, complex operations are performed, and circuits like flip-flops, counters, and processors are designed

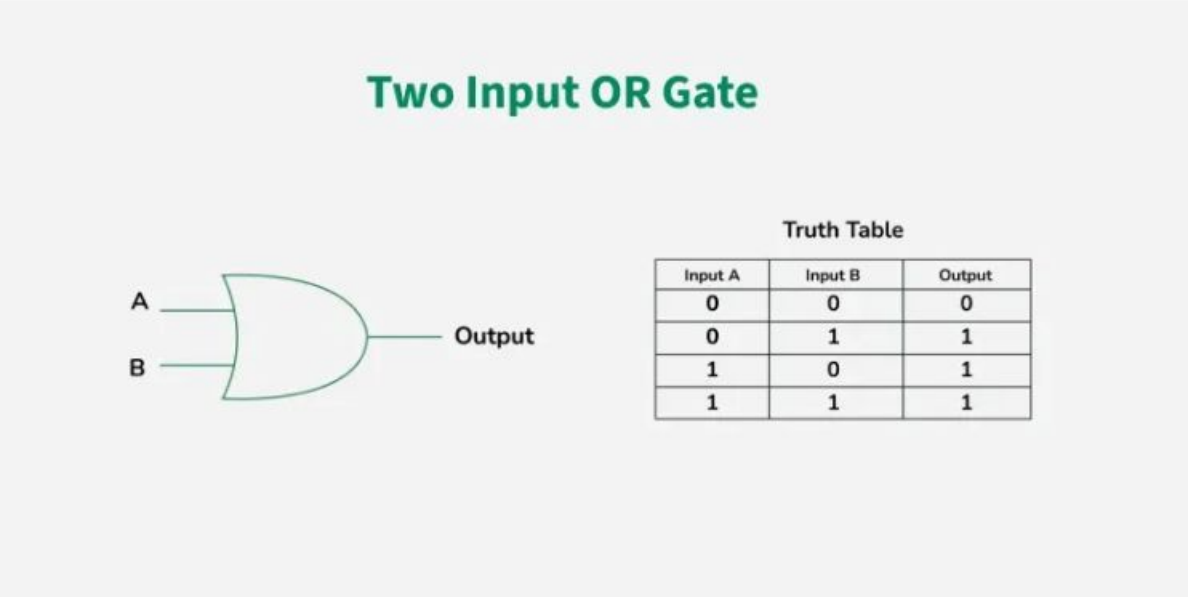
AND GATES

- AND gate is used to perform logical Multiplication of binary input
- The Output state of the AND gate will be high (1) if both the input is high (1), else the output state will be low(0) if any of the input is low (0).
- The Boolean Expression or logic for the AND gate is the logical multiplication of inputs denoted by a full stop or single dot as :
- A. B=X
- The value of X will be True when both the inputs will be True.



OR GATE

The OR gate is most widely used digital logic circuit. The output state of OR gate will be high i.e., (1) if any of the input state is high or 1, else output state will be low i.e., 0.
The Boolean Expression for the OR gate is the logical addition of inputs denoted by plus sign (+) as
X= A+B
The value of X will be high(true) when one of the inputs is set to high (true).



NOT GATE

- In digital electronics, the NOT gate is one of the basic Logic Gate having only a single input and a single output.
- It is also known as inverter or inverting buffer.
- When the input signal is "low" the output signal is "high" and vice-versa.
- The Boolean expression of NOT Gate is as follows
- $Y = \bar{A}$ or $Y = A'$
- the value of Y will be high when A will be low.

NOT GATE

Truth Table

A (Input)	Y = Ā (Output)
0	1
1	0

NOR GATE

The NOR gate is the type of universal logic gate. It takes two or more inputs and gives only one output. The output state of the NOR gate will be high (1) when all the inputs are low (0). NOR gate returns the complement result of the OR gate. It is basically a combination of two basic logic gates i.e., OR gate and NOT gate.

The Boolean expression of NOR gate is as follows:

If A and B are considered as two inputs, and Y as output, then the expression for a two input NOR gate will be $Y= (A + B)'$

The value of Y will be true when all of its inputs are set to 0.

NOR GATE

Two Input NOR Gate

Truth Table

Input A	Input B	0 = (A + B)'
0	0	1
0	1	0
1	0	0
1	1	0

NAND GATE

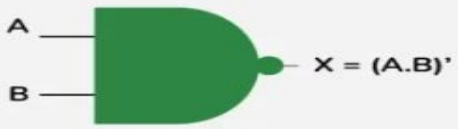
The NAND Gate is another type of Universal logic gate. The NAND gate or "Not AND" is the combination of two basic logic gates AND gate and the NOT gate connected in series.

It takes two or more inputs and gives only one output. The output of the NAND gate will give result high (1) when either of its input is high (1) or both of its input are low (0). In simple, it performs **the inverted operation of AND gate**.

The Boolean Expression of NAND Gate is as follows

Say we have two inputs, A and B and the output is called X, then the expression is X = (A. B)'

Two Input NAND Gate



Truth Table

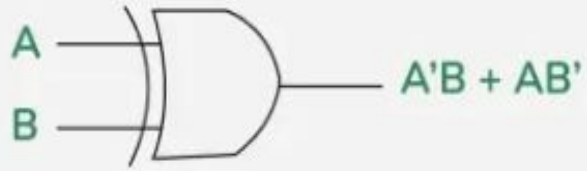
Input A	Input B	X = (A.B)'
0	0	1
0	1	1
1	0	1
1	1	0

XOR GATE

- In digital electronics, there is a specially designed logic gate named, XOR gate, which is used in digital circuits to perform modulo sum.
- It is also referred to as Exclusive OR gate or Ex-OR gate.
- It is used extensively in arithmetic logic circuits., logic comparators and error detection circuits.
- The XOR gate can take only two inputs at a time and give an output. The output of the XOR gate is high (1) only when its two inputs are dissimilar i.e., if one of them is low (0) then other one will be high (1).
- Say we have two inputs, A and B and the output is called X, then the expression is
- The Boolean expression of XOR Gate is $X = A'B + AB'$

XOR GATE

XOR Gate

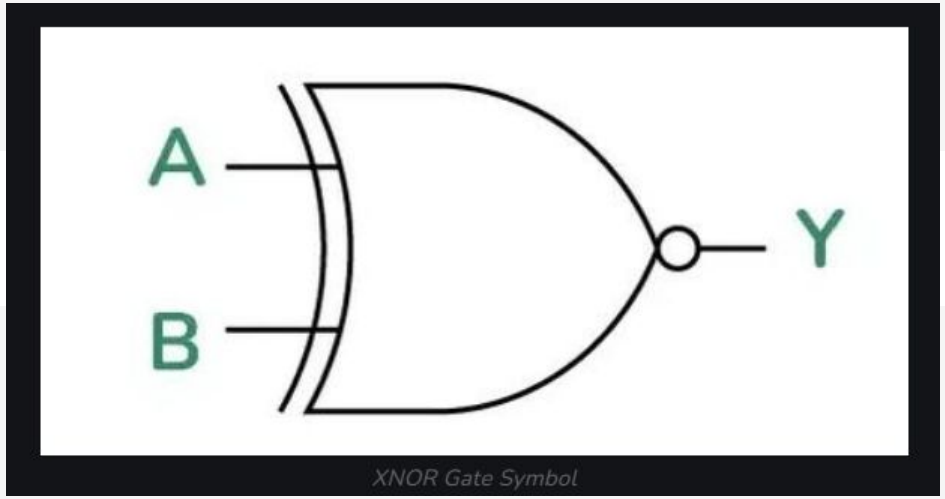


Truth Table

A (Input 1)	B (Input 2)	X = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

XNOR

An XNOR gate outputs a 1 (high) only when all its inputs are the same, and 0 (low) otherwise.



XNOR Gate Symbol

$Y = A \odot B$

$Y = AB + A'B'$

Truth Table

Input A	Input B	Output
0	0	1
0	1	0
1	0	0
1	1	1

Truth Table of XNOR Gate

SUM OF PRODUCT & PRODUCT OF SUM

SUM OF PRODUCT

$$Q = A.B.\overline{C} + A.\overline{B}.C + \overline{A}.B.C$$

Different product terms of input variables are logically ORed together.

PRODUCT OF SUM

$$Q = (A + B).(\overline{B} + C).(A + 1)$$

Different sum terms of input variables are logically ANDed together

Standard SOP Form

In SOP form, all the individual terms do not involves all the variables an literals. For example, in experts $F = ABC + \bar{B}C$ the second product term do not contain the variable A. If each term in SOP form contain all the variables then term in SOP form is known as standard (or) canonical SOP. Each terms in the standard SOP is called as minterms.

Example $F = \bar{A}BC + \bar{A}\bar{B}C + AB\bar{C}$

In the above example an the variables are present in the each term. It is the canonical form of SOP.

Standard POS Form

If each term in POS form contains all the variables, then POS is known as standard POS form (or) canonical form. The canonical POS form is also known as maxterms example is shown below

$$F = (A + \bar{B} + C) + (\bar{A} + B + \bar{C})$$

Converting Expressions in Standard SOP (or) POS Form

For a three variable experssion with variables A, B and C, if there is a term AB, where C is missing, them we form (C + C) and AND it with AB term.

$$F = A\bar{B}(C + \bar{C})$$
$$F = A\bar{B}C + A\bar{B}C \quad \ldots\ldots\ldots\ldots(1)$$

The equation(1) having all variables A, B, C then is is called canonical SOP (or) standard SOP. Similarly in POS form the missing variable is OR function with the given variables example if the function

$$F = (A + C) + (\bar{B} + \bar{C})$$
$$F = (A + C + B \cdot \bar{B})(\bar{B} + \bar{C} + A \cdot \bar{A})$$
$$F = (A + B + C)(A + \bar{B} + C)(A + \bar{B} + \bar{C})(\bar{A} + \bar{B} + \bar{C})$$

The final equation having all the variables, then it is called standard POS (or) canonical POS form.

PROBLEMS

Convert the standard SOP form for the function $F = \bar{B}C + \bar{A}\bar{C}$

☺ *Solution:*

$F = \bar{B}C(A+\bar{A}) + \bar{A}\bar{C}(B+\bar{B})$

$= A\bar{B}C + \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}\bar{B}C$

$F = A\bar{B}C + \bar{A}\bar{B}C + \bar{A}B\bar{C}$ [$\bar{A}\bar{B}C \rightarrow$ Omit repeated terms]

Convert to a canonical form of SOP for a function $F = AC + A\bar{B}\bar{C}$

☺ *Solution :*

$F = AC(B+\bar{B}) + A\bar{B}\bar{C}$

$F = ABC + A\bar{B}C + A\bar{B}\bar{C}$

Convert to a canonical form of POS for a function $F = (A+B+C)(\bar{A}+B)$

☺ *Solution :*

$F = (A+B+C)(\bar{A}+B)(C\cdot\bar{C})$

$F = (A+B+C)(\bar{A}+B+C)(\bar{A}+B+\bar{C})$

MINTERM & MAXTERM

Each individual terms in the standard SOP form is called as minterms and each individual term in standard POS form is called as maxterm. The minterms and maxterms for two variables is shown in the table 4.25 below

Table 4.25

Variables		Min terms	Max terms
x	y	m_i	M_i
0	0	$\overline{A}\,\overline{B} = m_0$	$A+B = M_0$
0	1	$\overline{A}\,B = m_1$	$A+\overline{B} = M_1$
1	0	$A\,\overline{B} = m_2$	$\overline{A}+B = M_2$
1	1	$AB = m_3$	$\overline{A}+\overline{B} = M_3$

Each minterms and maxterm represented by mi and M., where i is the decimal number equivalent of the natural binary number.

$$
\begin{aligned}
F(x,y,z) &= \overline{x}\,\overline{y}\,\overline{z} + \overline{x}\,y\,z + x\,y\,z \\
&= m_0 + m_3 + m_7 \\
F(x,y,z) &= \sum m(0, 3, 7) \\
F(x,y,z) &= (x+y+z)\,(x+y+\overline{z})\,(\overline{x}+y+z) \\
&= M_0 + M_1 + M_4 \\
F(x,y,z) &= \pi M(0, 1, 4)
\end{aligned}
$$

PROBLEMS

Convert the Boolean function to standard canonical form $F = \bar{A}\bar{B} + AC + \bar{B}C$

☺ ***Solution:***

$$F = \bar{A}\bar{B}(C + \bar{C}) + A(B + \bar{B})C + (A + \bar{A})\bar{B}C$$

$$= \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C} + ABC + A\bar{B}C + A\bar{B}C + \bar{A}\bar{B}C$$

Rearrange the terms

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}C + ABC$$

$$= m_0 + m_1 + m_5 + m_7$$

$$F(A, B, C) = \sum(0, 1, 5, 7)$$

Simplify the minterms $F(A, B, C) = \sum(0, 2, 4, 6)$

Solution :

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}\bar{C} + AB\bar{C}$$

$$F = \bar{B}\bar{C}(\bar{A} + A) + B\bar{C}(\bar{A} + A)$$

$$F = \bar{B}\bar{C} + B\bar{C} \qquad [\bar{A} + A = 1]$$

$$F = \bar{C}(\bar{B} + B)$$

$$F = \bar{C} \qquad [B + \bar{B} = 1]$$

Simplify the following three variable expression using Boolean algebra

$Y = \sum m(1, 3, 5, 6, 7)$

☺ ***Solution :***

$$Y = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

$$Y = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + AB(\bar{C} + C)$$

$$Y = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + AB \qquad [\bar{C} + C = 1]$$

$$Y = \bar{B}C(\bar{A} + A) + \bar{A}BC + AB$$

$$Y = \bar{B}C + \bar{A}BC + AB \qquad [\bar{A} + A = 1]$$

$$Y = \bar{B}C + B(\bar{A}C + A)$$

$$Y = \bar{B}C + B(A + C)$$

$$Y = \bar{B}C + AB + BC$$

$$Y = C(B + \bar{B}) + AB$$

$$\boxed{Y = AB + C} \qquad [B + \bar{B} = 1]$$

Convert the Boolean function to canonical form $F = AC + A\bar{B} + B$

☺ ***Solution :***

$$F = A(B + \bar{B})C + A\bar{B}(C + \bar{C}) + (A + \bar{A})B(C + \bar{C})$$

$$F = ABC + A\bar{B}C + A\bar{B}C + A\bar{B}\bar{C} + ABC + AB\bar{C} + \bar{A}B\bar{C}$$

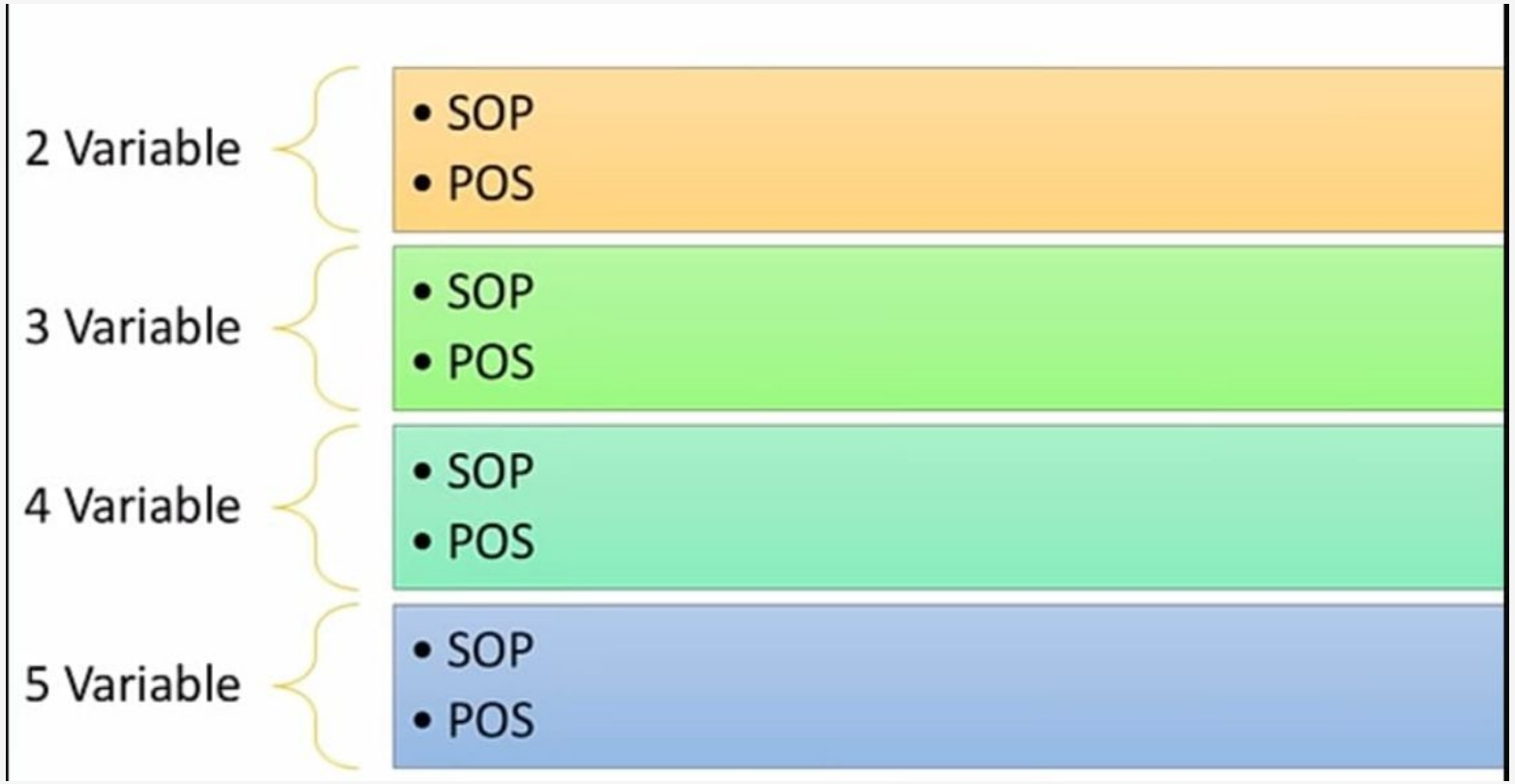
$$F = \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + AB\bar{C} + ABC$$

$$F = m_2 + m_3 + m_4 + m_5 + m_6 + m_7$$

$$F(A, B, C) = \sum(2, 3, 4, 5, 6, 7)$$

Karnaugh map K_MAP

A Karnaugh map (K-map) is a visual method for simplifying Boolean algebra expressions and digital logic circuits. It's a grid-based tool that helps identify patterns and minimize the number of logic gates needed.



Gray Coding

- Each cell within a K-map has a definite place value obtained using an encoding technique known as Gray code.
- The specialty of this code is the fact that the adjacent code values differ only by a single bit. That is, if the given code-word is 01, then the previous and the next code-words can be 11 or 00, in any order, but cannot be 10 in any case.
- In K-maps, the rows and the columns of the table use Gray code-labeling which in turn represents the values of the corresponding input variables. This means that each K-map cell can be addressed using a unique Gray Code-Word.
- These concepts are further emphasized by a typical 16-celled K-map shown in Figure 1, which can be used to simplify a logical expression comprising of 4-variables (A, B, C, and D mentioned in its top-left corner).

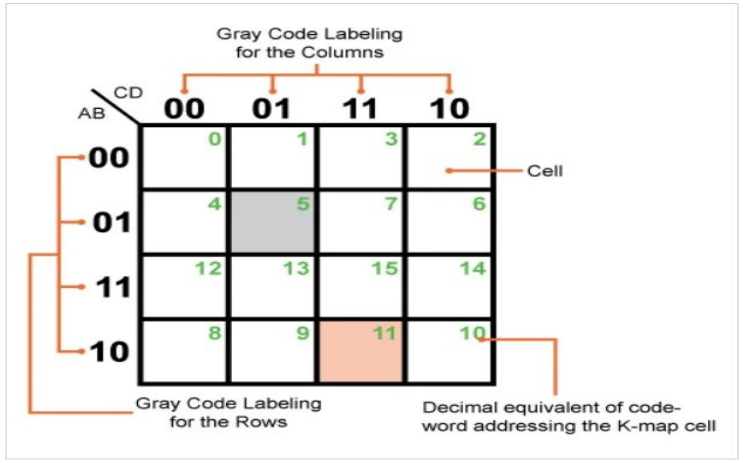


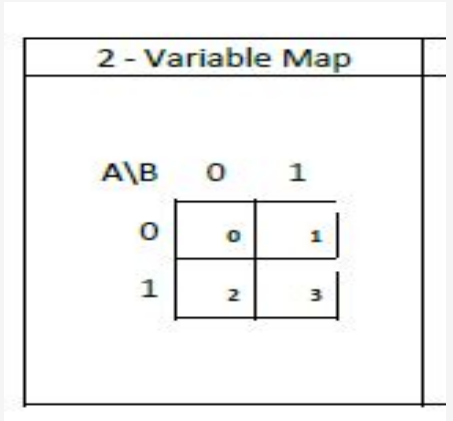
Figure 1. *A typical but empty Karnaugh map with 16 cells*

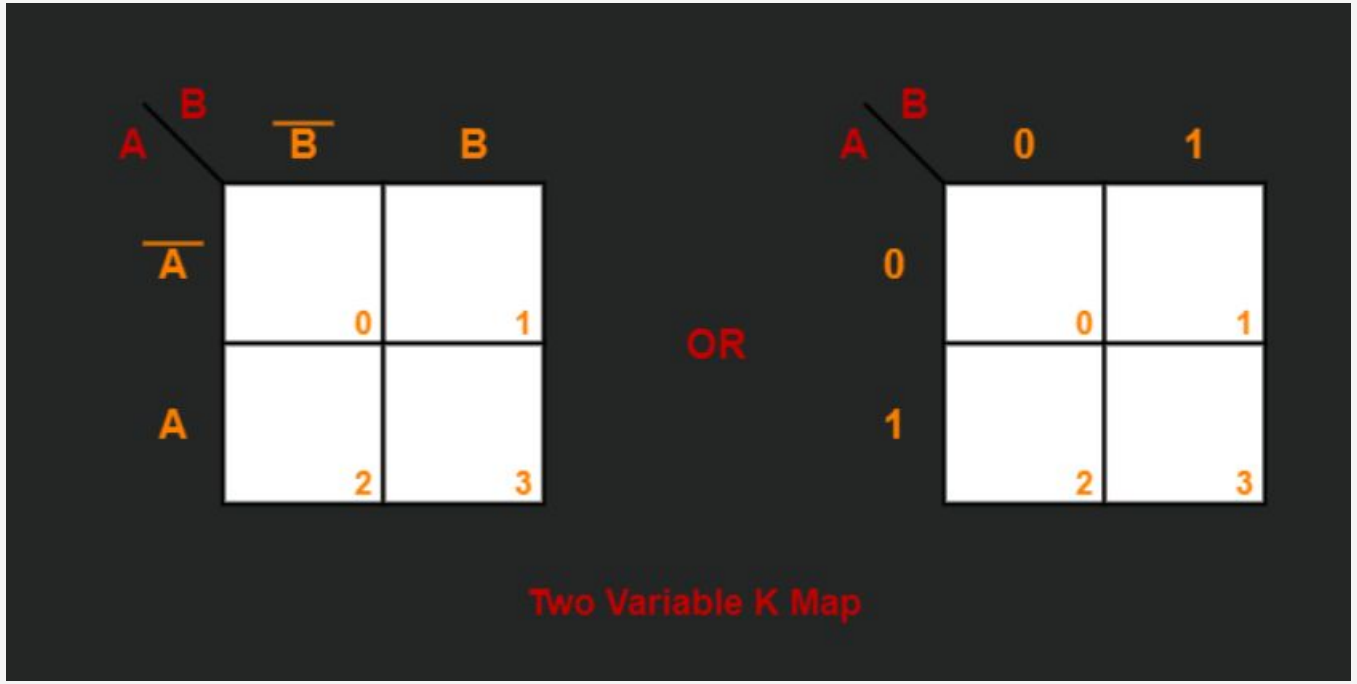
Two Variable K Map

Two variable K Map is drawn for a boolean expression consisting of two variables.

The number of cells present in two variable K Map = 2^2 = 4 cells.

So, for a boolean function consisting of two variables, we draw a 2 x 2 K Map.



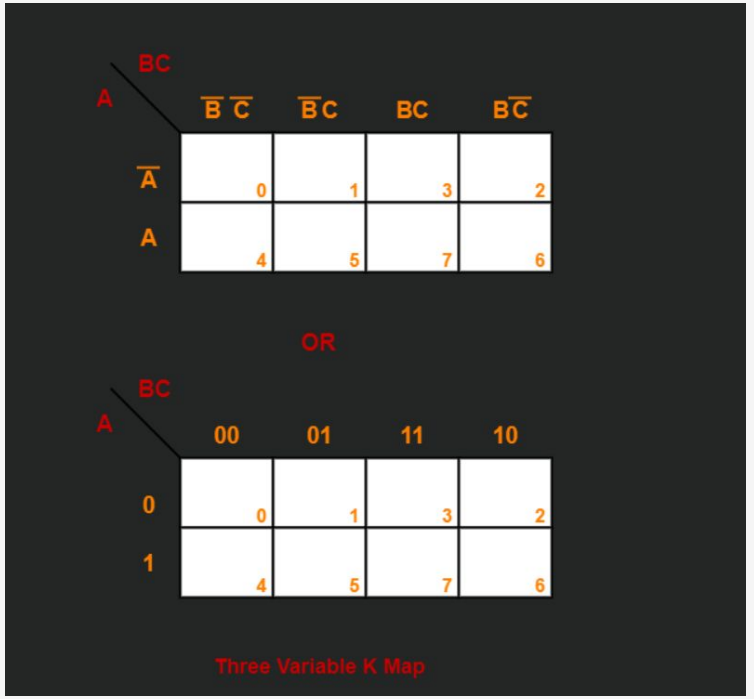


Three Variable K Map-

Three variable K Map is drawn for a boolean expression consisting of three variables.

The number of cells present in three variable K Map = 2^3 = 8 cells.

So, for a boolean function consisting of three variables, we draw a 2 x 4 K Map.

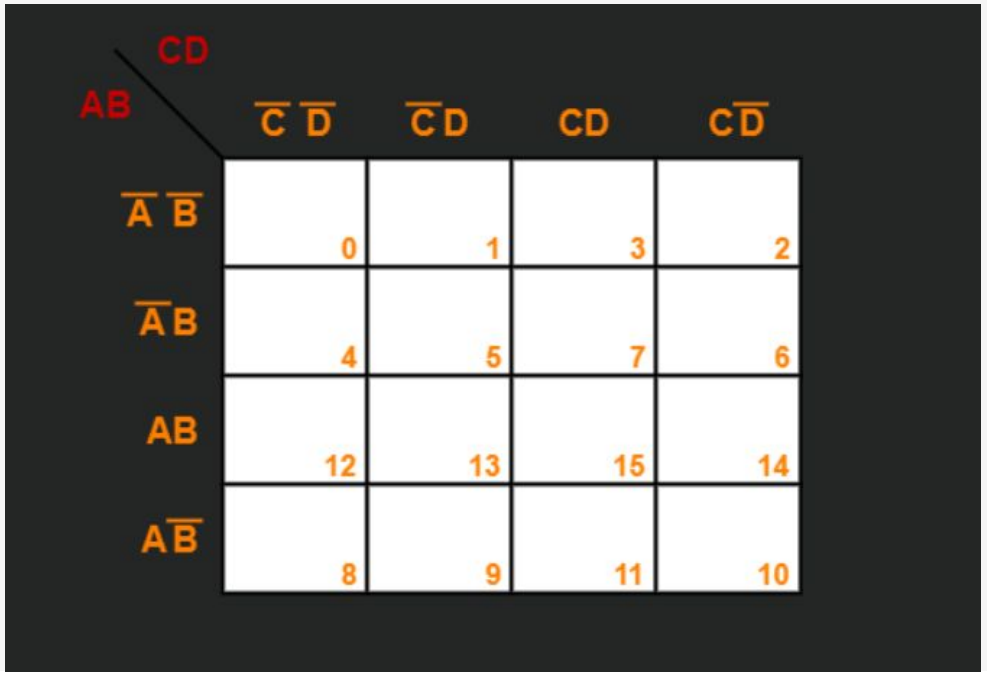


Four Variable K Map

Four variable K Map is drawn for a boolean expression consisting of four variables.

The number of cells present in four variable K Map = 2^4 = 16 cells.

So, for a boolean function consisting of four variables, we draw a 4 x 4 K Map.



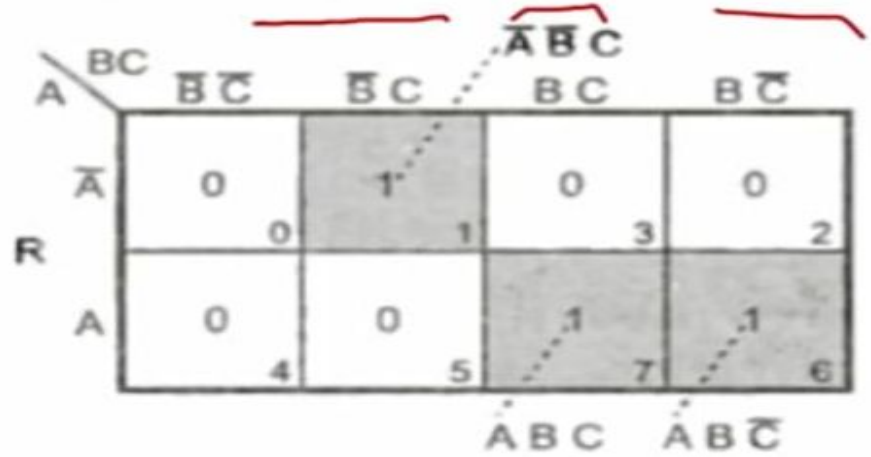
PROCEDURE FOR SIMPLIFICATIONS

The different rules for encircling the groups in the K-map have been discussed in the above sections. Now, using these rules, the method of getting the minimal Boolean expression of the given truth table (or function) will be summarized below:

- After forming the K-map, enter 1s for the min-terms corresponding to 1 in the truth table (or enter 1s for the min-terms of the given function to be simplified). Enter 0s for the remaining min-terms.
- Encircle octets, quads, and pairs, of course, remembering rolling and overlapping. Try to form groups of the maximum number of 1s.
- If any such 1s occur that are not used in any of the encircled groups, then these isolated 1s are encircled separately.
- Review all the encircled groups and remove the redundant groups, if any.
- Write the terms for each encircled group.
- The final minimal Boolean expression corresponding to the K-map will be obtained by ORing all the terms obtained above.

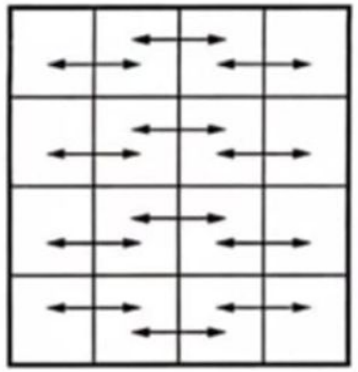
Representation of Boolean Expression on KMAP

Plot Boolean expression $Y = A\,B\,\overline{C} + A\,B\,C + \overline{A}\,\overline{B}\,C$ *on the Karnaugh map.*

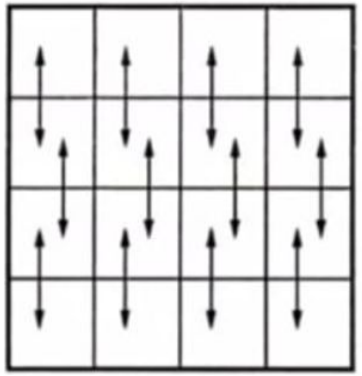


Karnaugh Map Simplification Rules

Neighbouring Cells



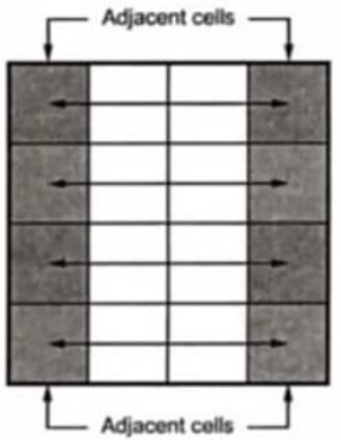
(a) Neighbouring cells in the row are adjacent



(b) Neighbouring cells in the column are adjacent

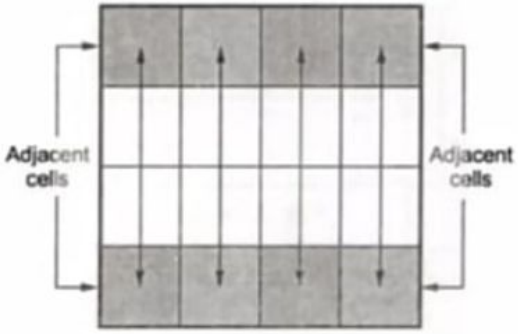
Neighbouring Cells

(a) Neighbouring cells in the row are adjacent



(c) Leftmost and corresponding rightmost cells are adjacent

(b) Neighbouring cells in the column are adjacent



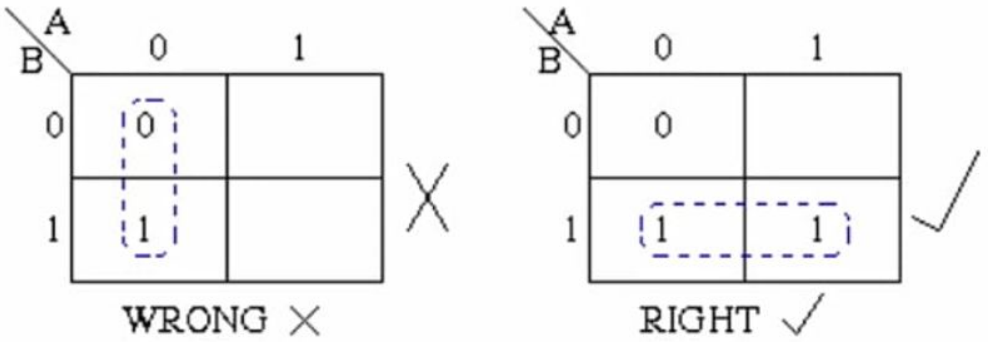
(d) Top and corresponding bottom cells are adjacent

RULES

- ✔ To minimize the given boolean function,
- ✔ We draw a K Map according to the number of variables it contains.
- ✔ We fill the K Map with 0's and 1's according to its function.
- ✔ Then, we minimize the function in accordance with the following rules.

RULE 1

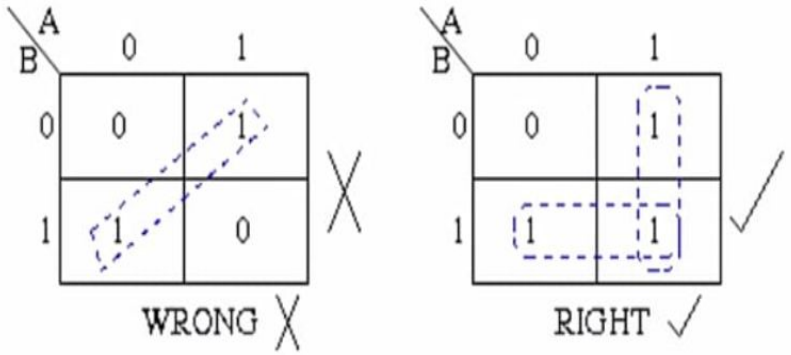
Groups may not include any cell containing a zero



RULES

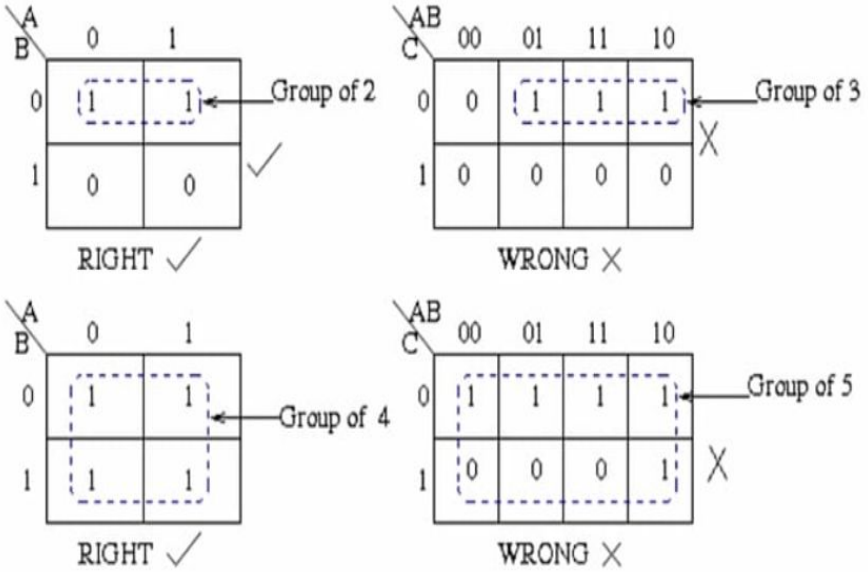
RULE 2

Groups may be horizontal or vertical, but not diagonal



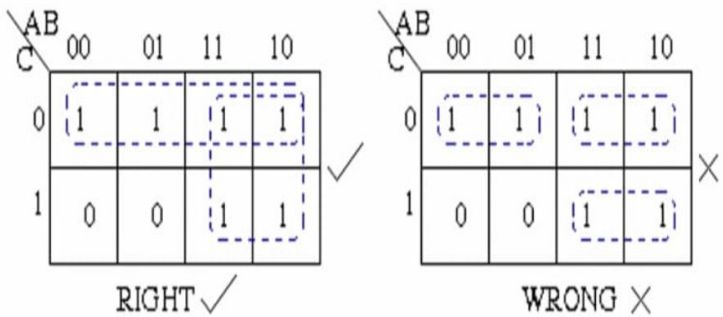
RULE 3

Groups must contain 1, 2, 4, 8,16 or in general 2^n cells.



RULE 4

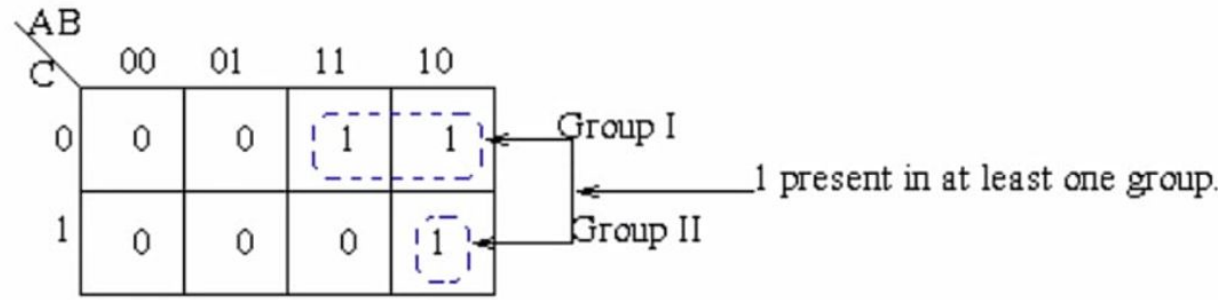
Each group should be as large as possible.



(Note that no Boolean laws broken, but not sufficiently minimal)

RULE 5

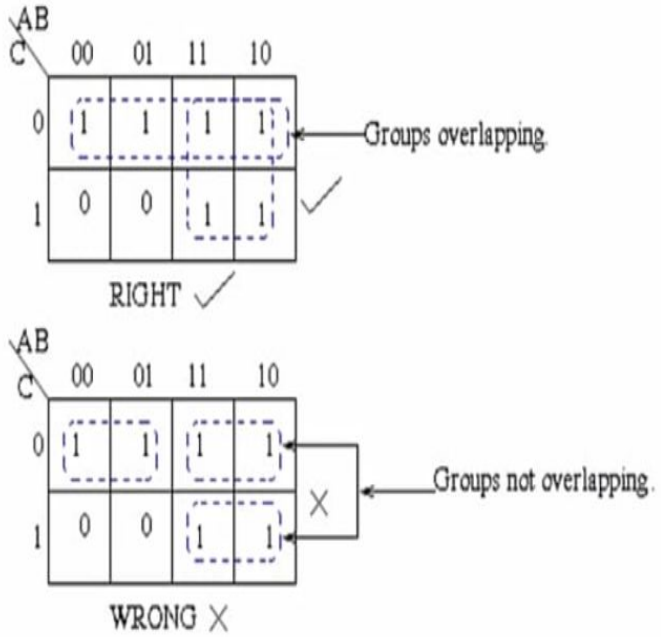
Each cell containing a one must be in at least one group.



RULES

RULE 6

Groups may overlap



RULES OF KMAP

The Karnaugh map uses the following rules for the simplification of expressions by *grouping* together adjacent cells containing *ones*

- No zeros allowed.
- No diagonals
- Only power of 2 number of cells in each group.
- Groups should be as large as possible.
- Every one must be in at least one group.
- Overlapping allowed.

DON'T CARE

- Sometimes, in a Boolean expression for certain input combinations, the value of the output is not specified either due to invalid input combinations or due to the precise value of output is of no importance.
- These input combinations for which the values of the Boolean function are not specified are called dont care combinations.
- **Dont care combinations** are also referred to as **optional combinations**. In K-map, dont care combinations are represented by "X" or "d"
- SOP (Sum of Products) K-map, each dont care term is treated as a 1, if it helps in reduction of the expression, otherwise it is considered as a 0 and left alone.
- POS (Product of Sums) K-map, each dont care term is considered as a 0, if it is helpful in reduction of the expression, otherwise it is treated as a 1 and left alone.

Don’t CARE

Minimize the following 4-variable Boolean expression in SOP form using K-map.

$$f(\mathrm{A},\mathrm{B},\mathrm{C},\mathrm{D}) = \sum \mathrm{m}(0,1,4,5,6,10,13) + \mathrm{d}(2,3)$$

Solution

The SOP K-map representation of the given Boolean function is shown in Figure 1.

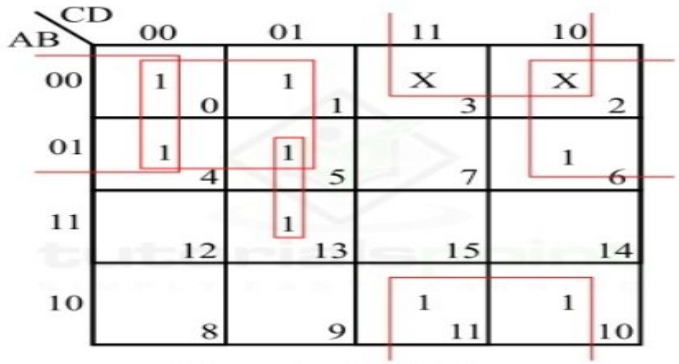


Figure 1 - SOP K-Map

Explanation

The reduction of the expression is done as per the following steps –

- The minterms m_0, m_1, m_4 and m_5 form a 4-square. Make it and read it as $(\bar{\mathrm{A}}\,\bar{\mathrm{C}})$.
- The minterms m_{10} and m_{11} form a 4-sqaure with two dont care terms m_2 and m_3. Make it and read it as $\mathrm{B}\bar{\mathrm{C}}$.
- The minterms m_0, m_4, m_6 and the dont care term m_2 together form a 4-square. Make it and read it as $\bar{\mathrm{A}}\,\bar{\mathrm{D}}$.
- The minterms m_5 and m_{13} form a 2-square. Make it and read it as $\mathrm{B}\bar{\mathrm{C}}\mathrm{D}$.
- Finally, write all the product terms in SOP form.

Therefore, the minimal Boolean expression is,

$$f(\mathrm{A},\mathrm{B},\mathrm{C},\mathrm{D}) = \bar{\mathrm{A}}\,\bar{\mathrm{C}} + \mathrm{B}\bar{\mathrm{C}} + \bar{\mathrm{A}}\,\bar{\mathrm{D}} + \mathrm{B}\bar{\mathrm{C}}\mathrm{D}$$

DON'T CARE

Minimize the following 4-variable Boolean expression in POS form using K map.

$$f(A,B,C,D) = \prod M(1,5,6,12,13,14) + d(2,4)$$

Solution

The POS K-map representation of the given Boolean function is shown in Figure-2.

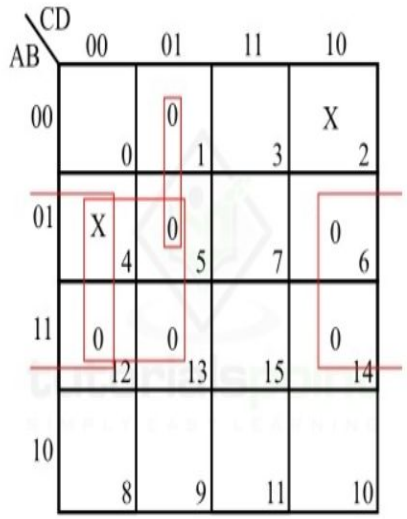


Figure 2 - POS K-Map

There are two dont care terms in the expression which are represented by X on the K-map.

Explanation

The reduction of the given function is done as per the following steps –

- The maxterms M_5, M_{12} and M_{13} and dont care term M_4 form a 4-square. Make it and read it as $\bar{B} + C$.
- The maxterms M_6, M_{12}, and M_{14} and dont care term M_4 form a 4-square. Make it and read it as $\bar{B} + D$.
- The maxterms M_1 and M_5 form a 2-square. Make it and read it as $A + C + \bar{D}$
- Write all the sum terms in POS form.

Therefore, the minimal Boolean expression is,

$$f(A,B,C,D) = (\bar{B} + C) + (\bar{B} + D) + (A + C + \bar{D})$$

5 variable K map

5 VARIABLE KMAP

- 5 Variable K-map requires 32 cells (2^5)
- The adjacent cells are difficult to identify on a single 32 cell K-map therefore two 16 cell K-maps are generally used
- If the variable are A,B,C,D & E two identical 16 cells maps containing B,C,D & E are constructed
- One map is used for A and other for $\bar{A}$

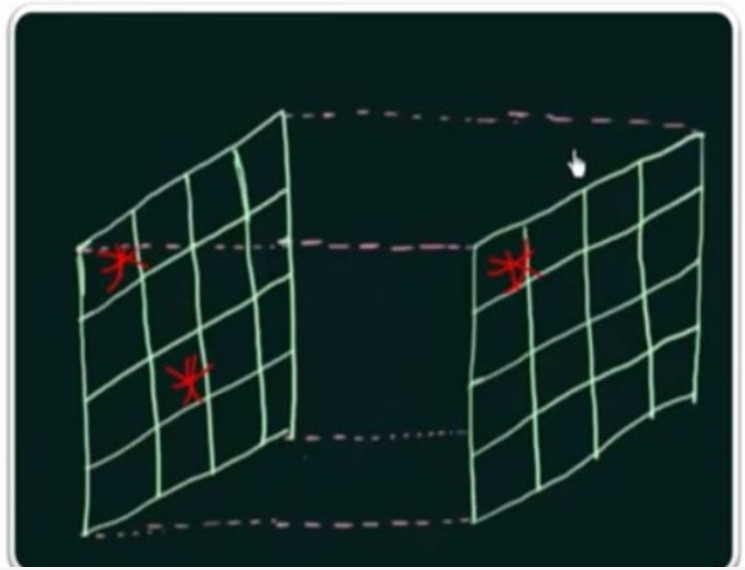
BLANK 5 VARIABLE KMAP

A = 0

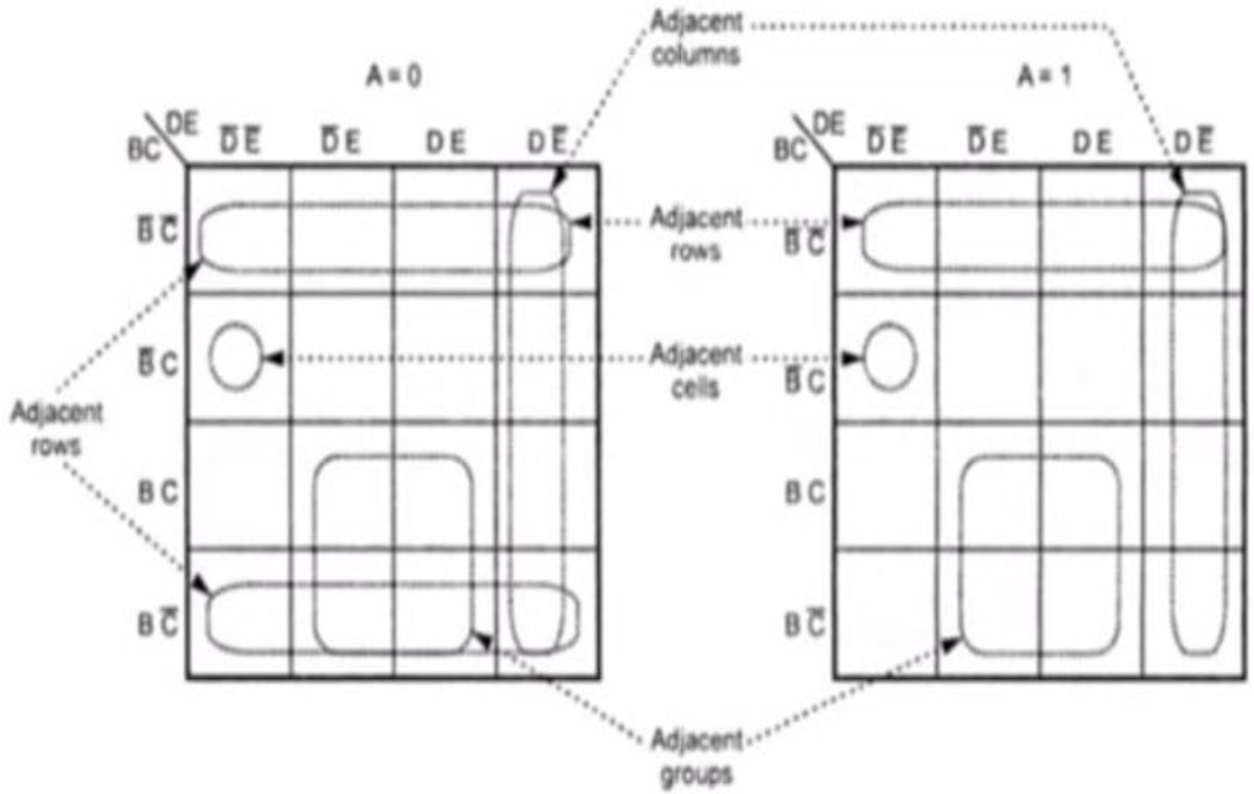
BC \ DE	$\bar{D}\bar{E}$ 00	$\bar{D}E$ 01	DE 11	$D\bar{E}$ 10
$\bar{B}\bar{C}$ 00	0	1	3	2
$\bar{B}C$ 01	4	5	7	6
BC 11	12	13	15	14
$B\bar{C}$ 10	8	9	11	10

A = 1

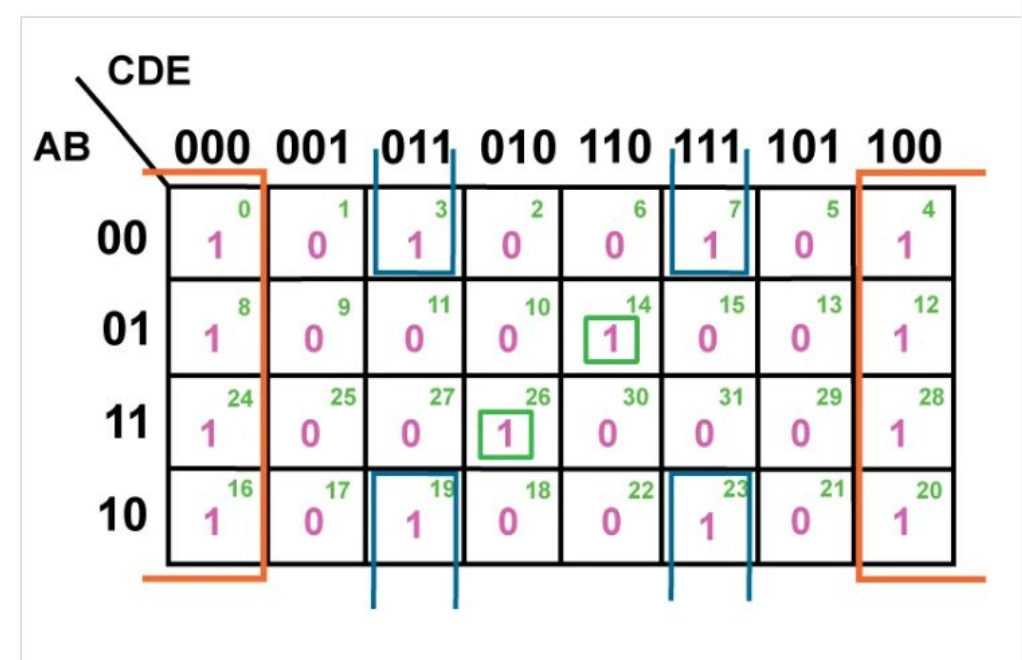
BC \ DE	$\bar{D}\bar{E}$ 00	$\bar{D}E$ 01	DE 11	$D\bar{E}$ 10
$\bar{B}\bar{C}$ 00	16	17	19	18
$\bar{B}C$ 01	20	21	23	22
BC 11	28	29	31	30
$B\bar{C}$ 10	24	25	27	26



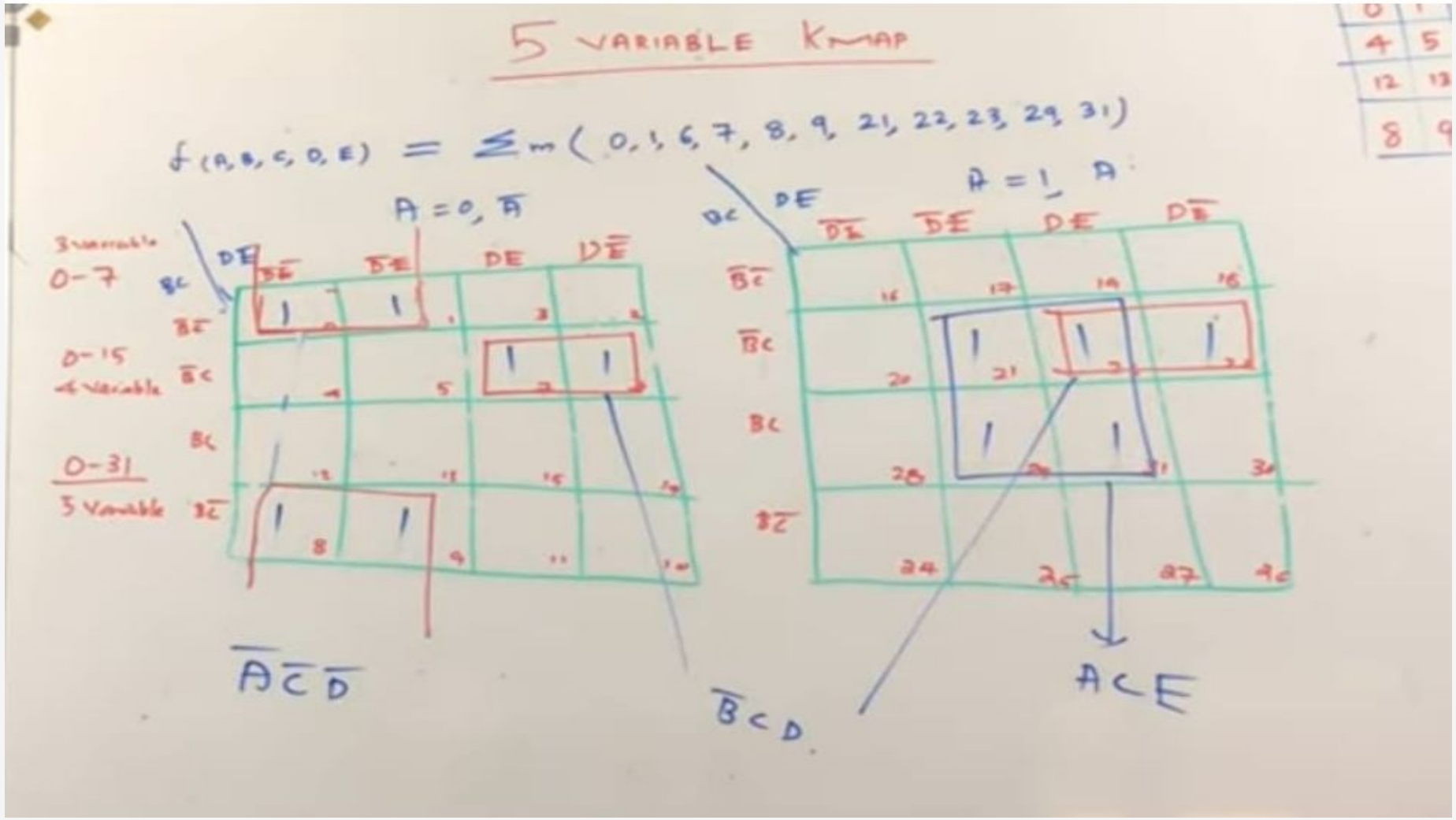
5 Variable KMAP With Example of Adjacencies

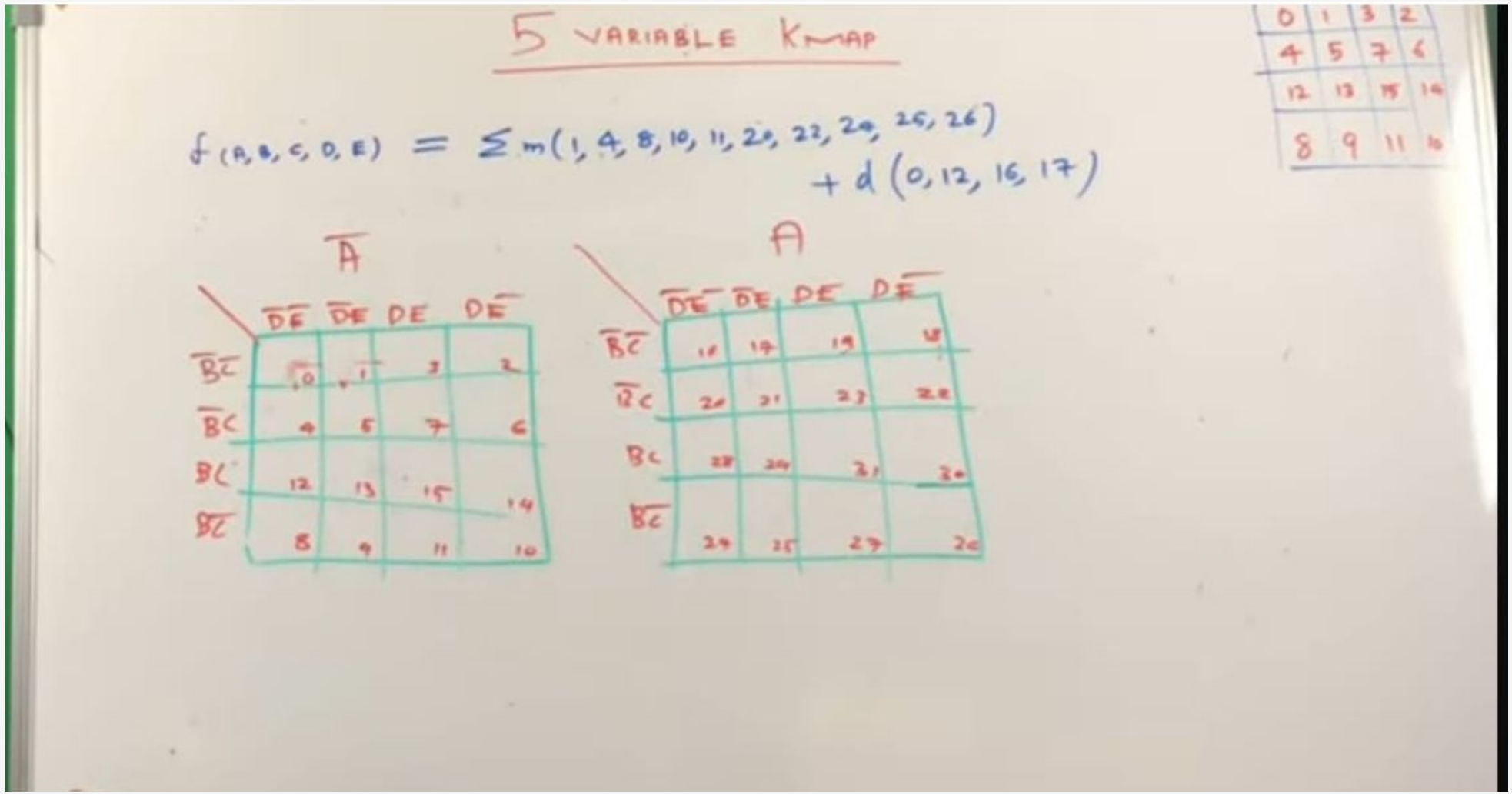


Q: Simplify the Boolean expression f (A,B,C,D,E) = $\sum m$ (0,3,4,7,8,12,14,16,19,20,23,24,26,28)



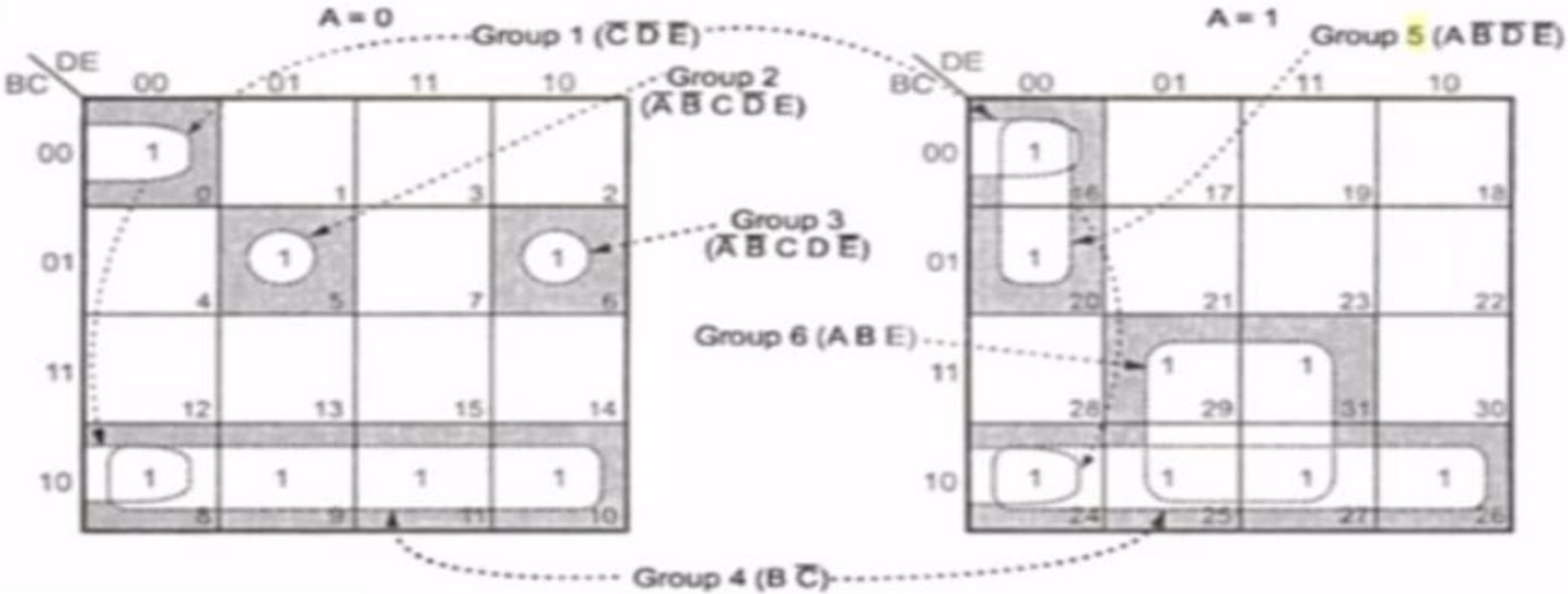
Thus, f (A,B,C,D,E) = $\bar{D}\bar{E} + \bar{B}DE + \bar{A}BCD\bar{E} + AB\bar{C}D\bar{E}$


5 VARIABLE KMAP
f(A,B,C,D,E) = Σm(0,1,6,7,8,9,21,22,23,29,31)
A = 0, Ā
A = 1, A
3 variable
0-7
0-15
4 Variable
0-31
5 Variable
DE
BC
ĀCD̄
BCD
ACE


5 VARIABLE K-MAP
f(A, B, C, D, E) = Σm(1, 4, 8, 10, 11, 20, 22, 29, 25, 26)
+ d(0, 12, 16, 17)
0 1 3 2
4 5 7 6
12 13 15 14
8 9 11 10
Ā
A
DE DE DE DE
BC
BC
BC
BC

Example 3.21 : *Simplify the Boolean function*

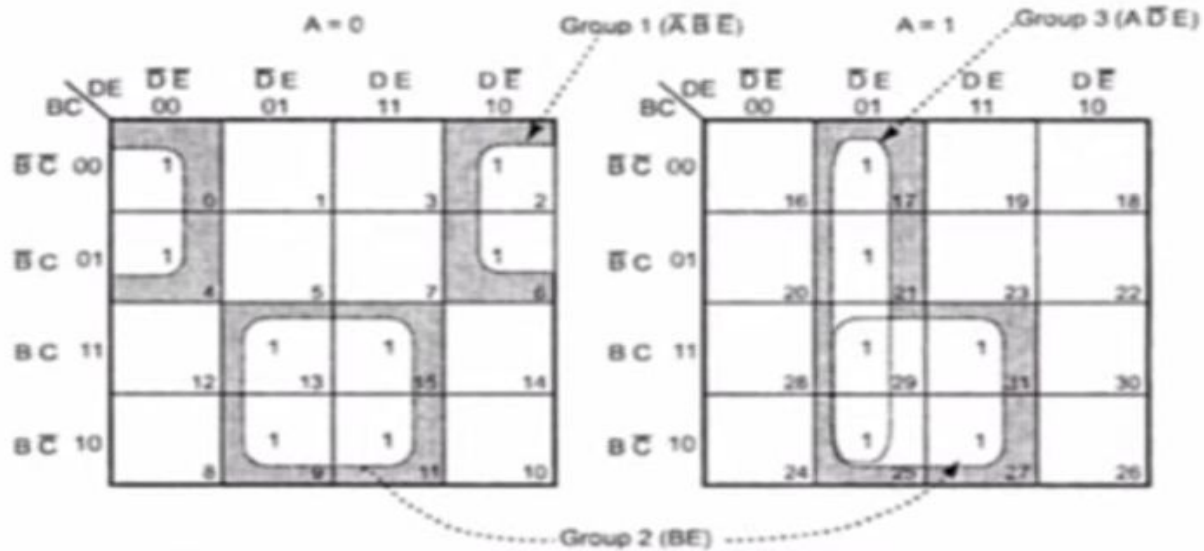
$$F(A, B, C, D, E) = \sum m\,(0, 5, 6, 8, 9, 10, 11, 16, 20, 24, 25, 26, 27, 29, 31)$$



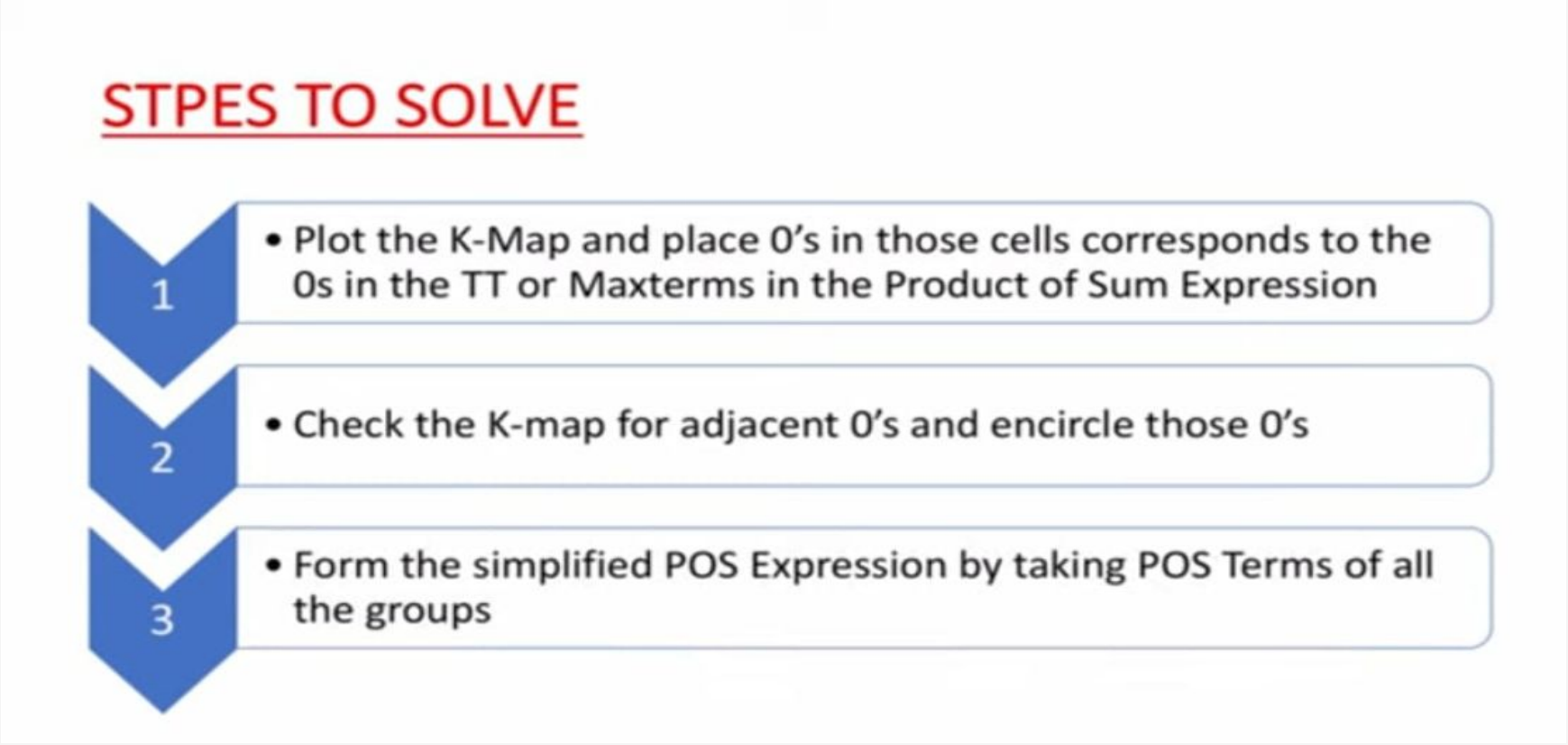
Example 3.20 : *Simplify the Boolean function.*

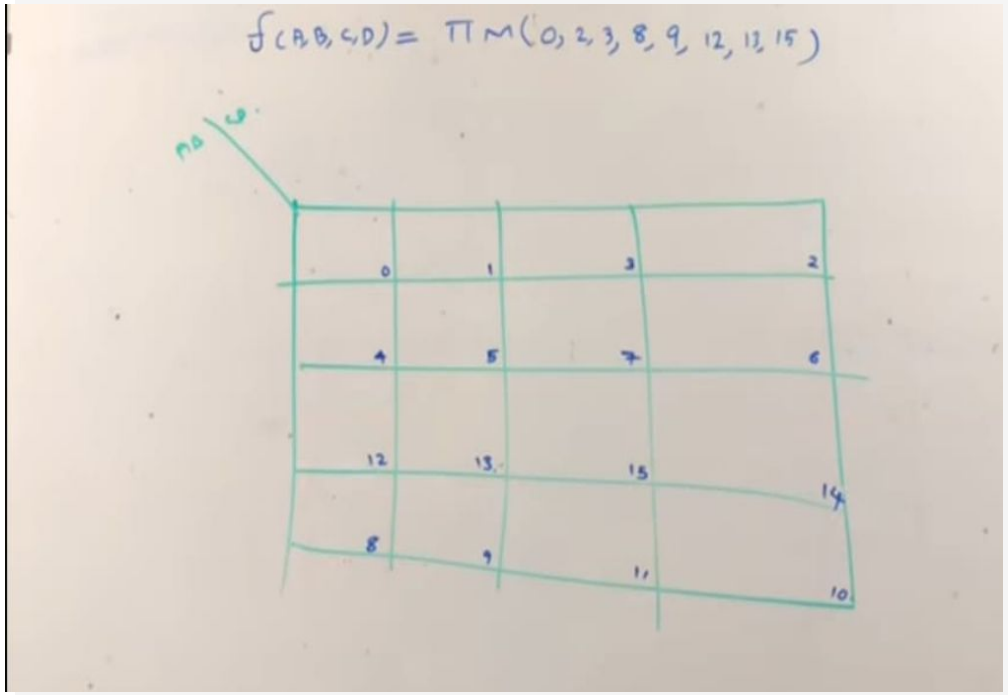
$$f(A, B, C, D, E) = \sum m(0, 2, 4, 6, 9, 11, 13, 15, 17, 21, 25, 27, 29, 31)$$

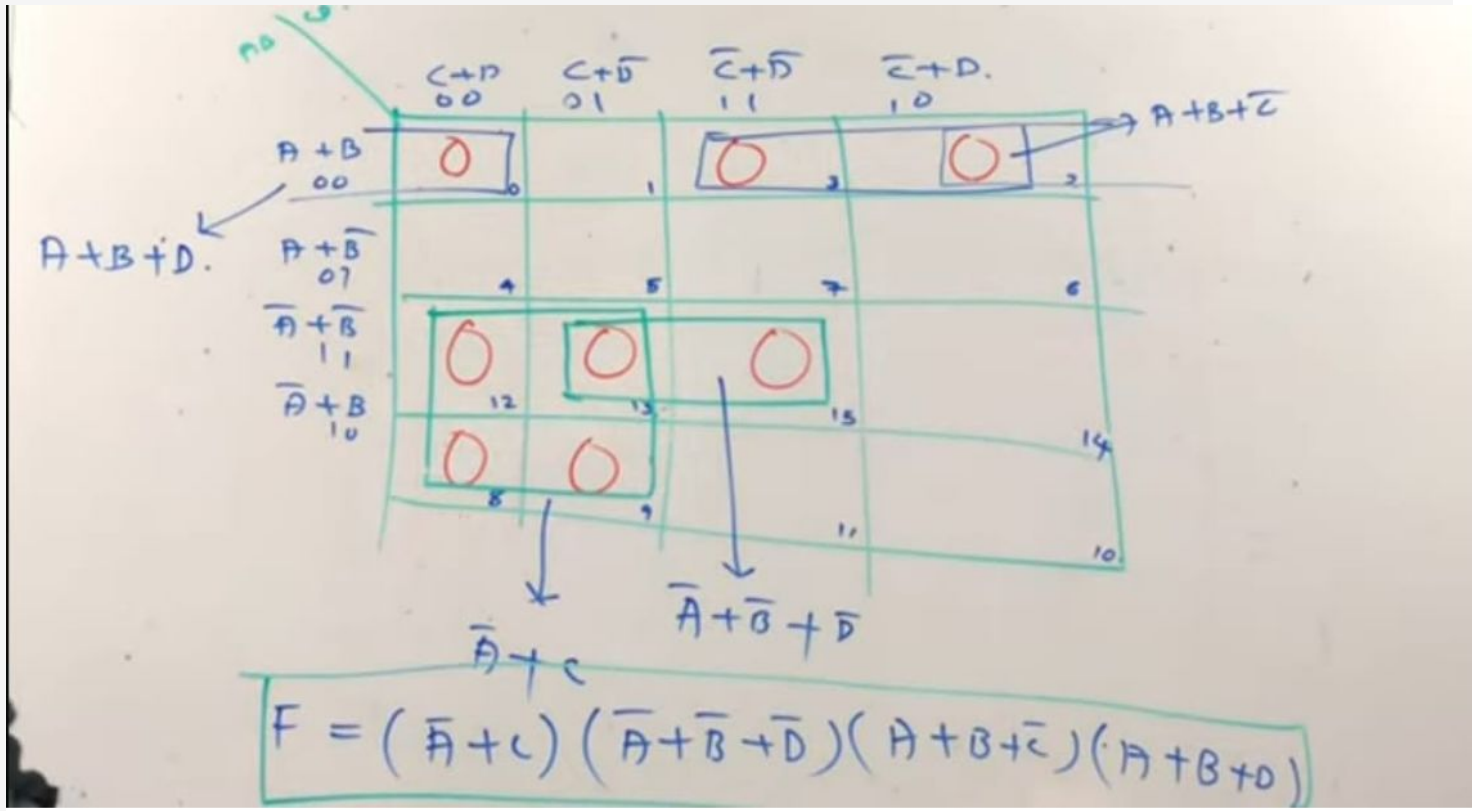
Solution :



PRODUCT OF SUM K–MAP




f(A,B,C,D) = Π M(0, 2, 3, 8, 9, 12, 13, 15)
AB
CD
0
1
3
2
4
5
7
6
12
13
15
14
8
9
11
10


C+D
00
C+D̄
01
C̄+D̄
11
C̄+D
10
A+B
00
A+B̄
01
Ā+B̄
11
Ā+B
10
A+B+C̄
A+B+D.
Ā+B̄+D̄
Ā+C
F = (Ā+C)(Ā+B̄+D̄)(A+B+C̄)(A+B+D)

Problems –K map

Example 1:

Design a digital system whose output is defined as logically low if the 4-bit input binary number is a multiple of 3; otherwise, the output will be logically high. The output is defined if and only if the input binary number is greater than 2.

Key: Step 1: Truth Table / Canonical Expression Leading to Min- or Max-Terms

Step 2 : Select and Populate K-Map

Step 3: Form the Groups – SOP and POS

Step 4: Simplified Logical Expression

SOP $Y = \bar{B}\bar{D} + \bar{A}\bar{C} + \bar{A}BD + B\bar{C}D + A\bar{B}C + AC\bar{D}$	POS $Y = (A+B)\,(B+C+\bar{D})\,(A+\bar{C}+D)\,(\bar{A}+\bar{B}+C+D)\,(\bar{A}+\bar{B}+\bar{C}+\bar{D})$

Step 5: System Design

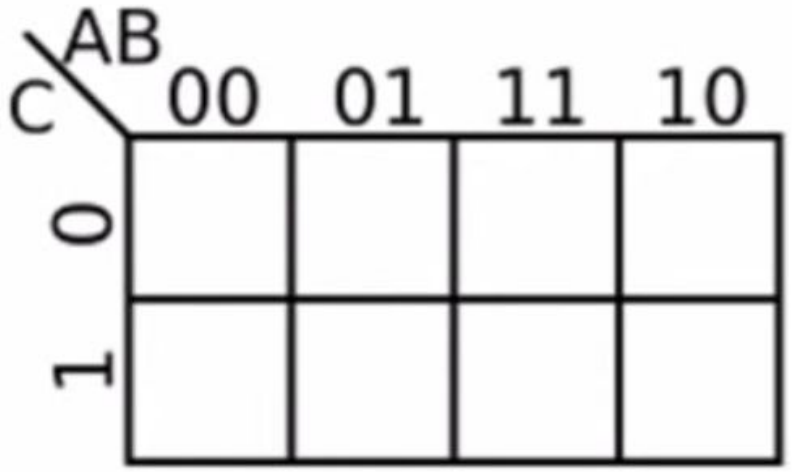
EXAMPLE 2

Example 2.21 : *Minimize the expression*

$$Y = (A + B + \overline{C})(A + \overline{B} + \overline{C})(\overline{A} + \overline{B} + \overline{C})(\overline{A} + B + C)(A + B + C)$$

Solution : $(A + B + \overline{C}) = M_1$, $(A + \overline{B} + \overline{C}) = M_3$, $(\overline{A} + \overline{B} + \overline{C}) = M_7$,

$(\overline{A} + B + C) = M_4$, $(A + B + C) = M_0$



EXAMPLE 3 & 4

Problems to workout

1

Using K-map simplify the Boolean function F as Sum of Products using the don't care conditions d.

$F(w,x,y,z)=w'(x'y+ x'y' +xyz) + x'z'(y+w)$

$d(w,x,y,z)=w'x(y'z + yz) +wyz$

2

Reduce the following expressions using K-map and implement the real minimal expression in universal logic.

1) $F(A,B,C,D)=\sum m(0,1,2,3,5,7,8,9,10,12,13)$

EXAMPLE 5,6 &7

Problems to workout

3 b) Simplify the given Boolean function F (w, x, y, z) =∑(2, 3, 12, 13, 14, 15)
i) Sum of Products and ii) Product of Sums (use K Map)

4 Simplify the Boolean function F (w, x, y, z) = ∑m(0, 5, 7, 8, 9, 10, 11, 14, 15)

5 **Example 3.10 :** *Reduce the following function using Karnaugh map technique and implement using basic gates*

$f(A, B, C, D) = \overline{A}\overline{B}D + AB\overline{C}\overline{D} + \overline{A}BD + ABC\overline{D}$

EXAMPLE 8 & 9

Minimize the following function using K-map and realize it with AND, OR & NOT logic gates.

$$X(A, B, C, D,) = \Sigma(0, 1, 2, 5, 8, 10, 11, 14, 15)$$

Minimize the following boolean function-

$$F(A, B, C, D) = \Sigma m(0, 1, 2, 5, 7, 8, 9, 10, 13, 15)$$

Ans: BD + C'D + B'D'

ASSIGNMENT 1

Smart Energy Management System Design

Background: Design a logic controller for a smart building's energy management system that automatically controls lighting, HVAC, and electrical appliances based on occupancy, time of day, and energy availability from renewable sources. This system addresses the challenge of reducing energy consumption while maintaining comfort and accessibility for all building occupants.

Problem Statement: Your energy management controller receives the following inputs:

A: Room Occupied (1 = occupied, 0 = vacant)

B: Daylight Available (1 = sufficient natural light, 0 = artificial light needed)

C: Peak Energy Hours (1 = peak demand period, 0 = off-peak)

REQUIRMENTS

Tasks:

Part A: Boolean Expression Development

- Write Boolean expressions for L, H, and P based on the requirements
- Create truth tables for all three outputs (16 combinations each)
- Verify your expressions by testing 3 specific scenarios

PART B: K-map Optimization

- Draw 4-variable K-maps for each output function
- Identify optimal groupings and derive minimized SOP expressions
- Calculate the reduction in gate count achieved through optimization
- Draw the optimized circuit diagrams using basic gates

Part C: Implementation Considerations

1. Convert your optimized design to use only NAND gates
2. Analyze the power consumption implications of your optimization
3. Suggest how this controller could be integrated with IoT systems for smart city networks
4. Suggest one ethical consideration in automated building control systems